

SISTEMAS OPERACIONAIS

V - Sincronização

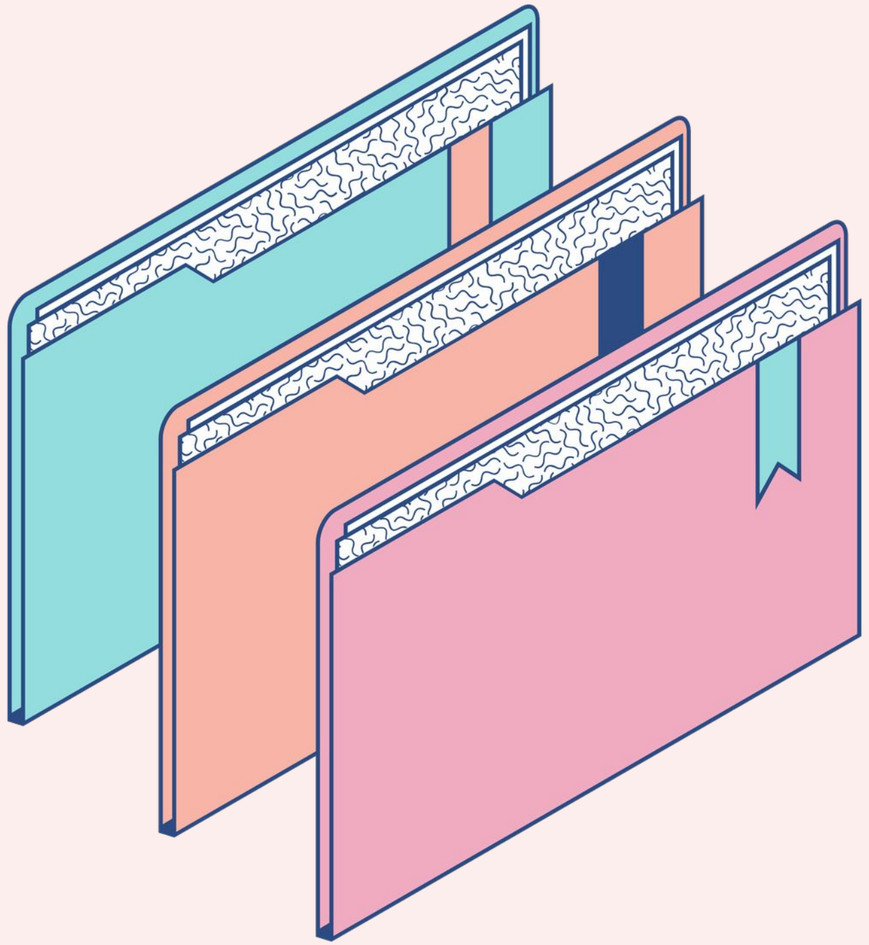
Objetivo Central:

Coordenar a execução de múltiplos processos/threads concorrentes que compartilham recursos, garantindo a consistência dos dados.

💡 **Exemplo Prático:**

Dois saques bancários simultâneos na mesma conta. Sem sincronização, ambos leem o mesmo saldo inicial, causando erro de cálculo (Condição de Corrida).

Professor: PhD. Eduardo Duarte



Agenda da Aula

1. Recapitulação: Threads

Revisão de processos leves e compartilhamento de memória.

2. Antecedentes da Sincronização

Necessidade de ordem no acesso concorrente a recursos.

3. O Problema da Seção Crítica

Evitando condições de corrida (Ex: saques simultâneos).

4. Solução de Peterson

Algoritmo clássico de exclusão mútua em software.

5. Semáforos e Monitores

Mecanismos de alto nível para controle de acesso.

6. Sincronização na Prática

Exemplos reais e primitivas em Sistemas Operacionais.

Unidade IV - Threads



O que é uma Thread?

💡 Conceito Fundamental

Uma **thread** é a **menor unidade de execução** enviada à CPU. É o "fluxo de controle" dentro de um programa.

Componentes Próprios da Thread:

1. ID da Thread
Identificador único no SO.

2. Contador (PC)
Indica a próxima instrução.

3. Registradores
Variáveis de trabalho atual.

4. Pilha (Stack)
Histórico de funções e dados.

⚙️ Processo vs. Múltiplas Threads

📦 **PROCESSO (Compartilha Código, Dados e Recursos do SO)**

Thread 1
- Registers
- Stack
- PC

Thread 2
- Registers
- Stack
- PC

Thread 3
- Registers
- Stack
- PC

📌 Por que usar Múltiplas Threads?

Sistemas modernos permitem que um único processo execute **várias tarefas em paralelo**.

🌐 Exemplo Prático (Navegador Web):

- **Thread 1:** Renderiza o texto da página.
- **Thread 2:** Baixa imagens da rede em segundo plano.
- **Thread 3:** Responde aos cliques do usuário.



Diferença entre Concorrência e Paralelismo

⚡ 1. Concorrência (Lidar com várias coisas)

Conceito: O sistema dá suporte a múltiplas tarefas **progredindo** ao mesmo tempo, alternando rapidamente entre elas (troca de contexto).

Exemplo: Um garçom atendendo várias mesas sozinho.

Diagrama: 1 Único Núcleo (CPU Core)



- Execução **intercalada** (tempo compartilhado).
- Dá a **ilusão** de simultaneidade.
- Funciona mesmo em hardware de **1 único núcleo**.

🚀 2. Paralelismo (Fazer ao mesmo tempo)

Conceito: O sistema executa duas ou mais tarefas **exatamente no mesmo instante físico** de tempo.

Exemplo: Dois garçons atendendo mesas simultaneamente.

Diagrama: Múltiplos Núcleos (Multi-Core)



- Execução **simultânea real** de instruções.
- **Exige hardware dedicado** (CPUs/Cores múltiplos).

💡 **Conclusão Chave para Aprendizado:**

Concorrência é sobre a **estrutura do software** (lidar com muitas coisas). Paralelismo é sobre a **execução no hardware** (fazer muitas coisas juntas). Logo: **É possível ter Concorrência SEM Paralelismo!**



Lei de Amdahl: Ganho de Desempenho Potencial

📐 O Conceito & a Fórmula

A **Lei de Amdahl** calcula o ganho máximo de velocidade (speedup) ao adicionar mais núcleos de processamento a uma aplicação.

$$aceleração \leq \frac{1}{S + \frac{(1-s)}{N}}$$

Definição dos Componentes:

- **S (Fração Serial):** Parte do programa que deve rodar em 1 único núcleo (não paralelizável).
- **(1 - S) (Fração Paralela):** Parte do programa que pode ser dividida entre os núcleos.
- **N (Núcleos):** Quantidade total de núcleos de processamento (cores) disponíveis.

💡 Exemplo Prático & Aplicação

Suponha um sistema onde **25% do código é serial (S = 0,25)** e 75% pode ser paralelizado:

Com N = 5 Núcleos

~2,5x

Aceleração em relação a 1 núcleo

Com N = ∞ Núcleos

4,0x (Límite)

Limite teórico inultrapassável

📌 Conclusão Chave para Aprendizado:

- A componente serial (**S**) cria um **gargalo intransponível** para o ganho de desempenho.
- Adicionar infinitos processadores não reduz o tempo a zero, pois a parte serial sempre levará o mesmo tempo.
- Para melhorar a aceleração, a prioridade do desenvolvedor deve ser **diminuir a fração serial (S)**.

Parte I - Antecedentes da Sincronização



Processos Cooperativos: Conceito e Comunicação

📌 O Conceito Principal

Um **processo cooperativo** é aquele que pode **afetar** ou **ser afetado** por outros processos em execução no sistema.

⚠️ O Grande Desafio: Inconsistência

O acesso simultâneo (concorrente) a dados compartilhados sem sincronização adequada pode causar **corrupção e inconsistência de dados**.

🔄 Como Eles Se Comunicam?

1. Memória Compartilhada

Processos compartilham diretamente um **espaço de endereçamento lógico** (código e variáveis na RAM).

⚡ **Vantagem: Alta velocidade de comunicação.**

2. Troca de Mensagens

Comunicação mediada pelo Sistema Operacional por meio de **arquivos** ou **mensagens**.

🛡️ **Vantagem: Maior isolamento e segurança.**

Antecedentes: Concorrência e Comunicação

⚠️ Concorrência & Dados

Integridade em Risco

A execução **concorrente ou paralela** pode afetar a integridade de dados compartilhados.

Exemplo Prático:

Se dois processos tentam atualizar o mesmo saldo bancário ao mesmo tempo, o resultado final pode ser inconsistente sem sincronização.

🔄 Produtor-Consumidor

O Cenário Clássico

Processos/threads cooperativos executando **assincronamente** e compartilhando dados.

⚙️ **Produtor:** Gera informações/itens.

📥 **Consumidor:** Processa/consome os itens gerados.

📦 Buffer Limitado

Solução de Memória

Um **buffer limitado** habilita processos a compartilharem memória com capacidade fixa.

Como Funciona:

Funciona como uma fila intermediária: o produtor insere itens no buffer e o consumidor os retira sequencialmente.



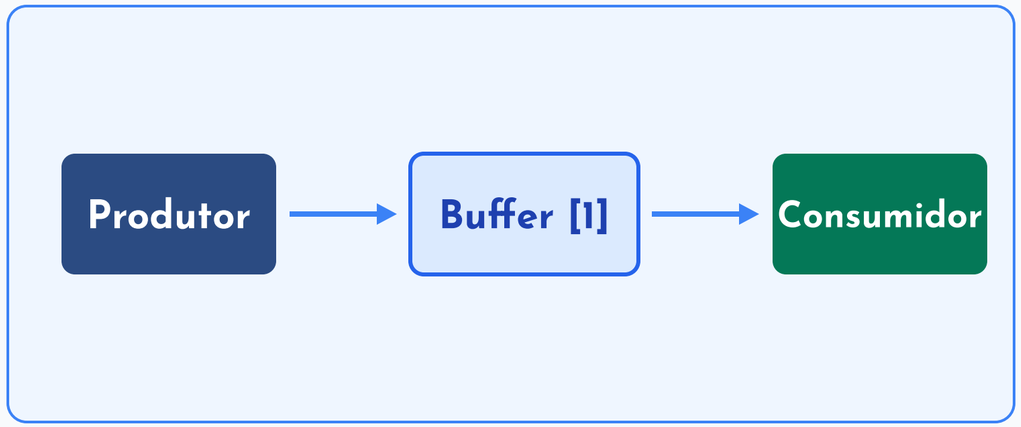
Antecedentes: Capítulos do Buffer em Sistemas

1. Problema Simples (Buffer_Size = 1)

Capacidade Estrita: O Buffer possui apenas **1 única posição** de memória disponível.

Fluxo de Execução: O processo **Produtor** insere um item e fica obrigatoriamente aguardando (bloqueado) até que o **Consumidor** retire este item.

💡 **Exemplo Prático:** Como uma passagem de bastão direta em uma corrida — o próximo atleta só corre após receber o bastão.

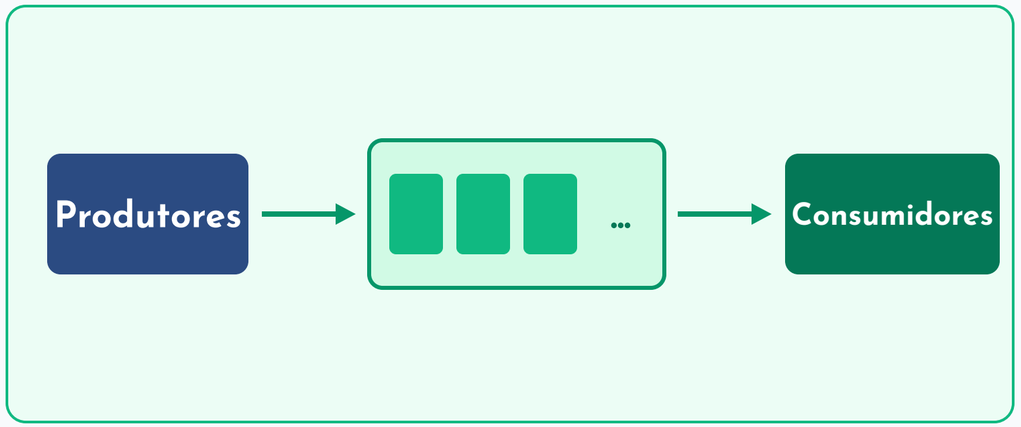


2. Sistemas Complexos (Buffer_Size > 1)

Alta Concorrência: Múltiplas tarefas sendo executadas em paralelo exigem maior vazão e desacoplamento.

Fila Intermediária: O Buffer armazena **múltiplos itens**, permitindo que o Produtor continue gerando dados sem travar imediatamente.

💡 **Exemplo Prático:** Como uma esteira de fábrica com várias caixas — o produtor empilha enquanto o consumidor retira no seu próprio ritmo.



Antecedentes: A Variável Counter

🔢 A Solução: Counter

Conceito Chave

Adiciona-se uma variável inteira **counter**, inicializada com **0**.

Objetivo

Rastrear em tempo real a quantidade exata de itens armazenados no buffer.

🔄 Dinâmica das Operações

Como Funciona

⚙️ **Produtor:**
Sempre que produz e insere um item no buffer, ele **incrementa** o counter (counter++).

📥 **Consumidor:**
Sempre que retira e consome um item, ele **decrementa** o counter (counter--).

🏭 Exemplo Prático

Analogia do Depósito

Pense no **counter** como um placar digital em um armazém de caixas:

+1 Caixa Entrou:
O produtor atualiza o placar para refletir +1.

-1 Caixa Saiu:
O consumidor atualiza o placar para refletir -1.

Evita que o armazém trasborde ou esvazie sem aviso!



Antecedentes: O Processo Produtor

💻 Algoritmo do Produtor

```
while (true) {

/* 1. Produz o item */

item = produzir_item();

/* 2. Teste de Buffer Cheio */

while (counter == BUFFER_SIZE)
; /* Espera ocupada */

/* 3. Insere no Buffer */

buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;

/* 4. Incrementa contador */

counter++;

}
```

📘 Explicação Passo a Passo

1. Gerar Dado: O processo fabrica um item localmente antes de tentar enviá-lo.

2. Teste de Lotação: Se **counter == BUFFER_SIZE**, o buffer está totalmente cheio. O produtor fica travado no loop repetições sem fazer nada.

3. Inserção Circular: Coloca o item no índice **in** e incrementa de forma circular usando o operador módulo **(% BUFFER_SIZE)**.

4. Atualiza Sinal: Executa **counter++** para indicar que há mais um item pronto.

💡 Exemplo Prático:

Como um cozinheiro colocando pratos numa prateleira: se a prateleira enche (BUFFER_SIZE), ele precisa esperar até o garçom retirar algum item.



Antecedentes: O Processo Consumidor

📥 Algoritmo do Consumidor

```
while (true) {

/* 1. Teste de Buffer Vazio */

while (counter == 0)
; /* Espera ocupada */

/* 2. Retira do Buffer */

next_consumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;

/* 3. Decrementa contador */

counter--;

/* 4. Consome o item */

consome_item(next_consumed);

}
```

📘 Explicação Passo a Passo

1. Teste de Vazio: Se **counter == 0**, não há itens disponíveis. O consumidor aguarda em loop (espera ocupada).

2. Remoção Circular: Retira o item do índice **out** e incrementa a posição circularmente (**% BUFFER_SIZE**).

3. Atualiza Sinal: Executa **counter--** para registrar que um item foi retirado do buffer.

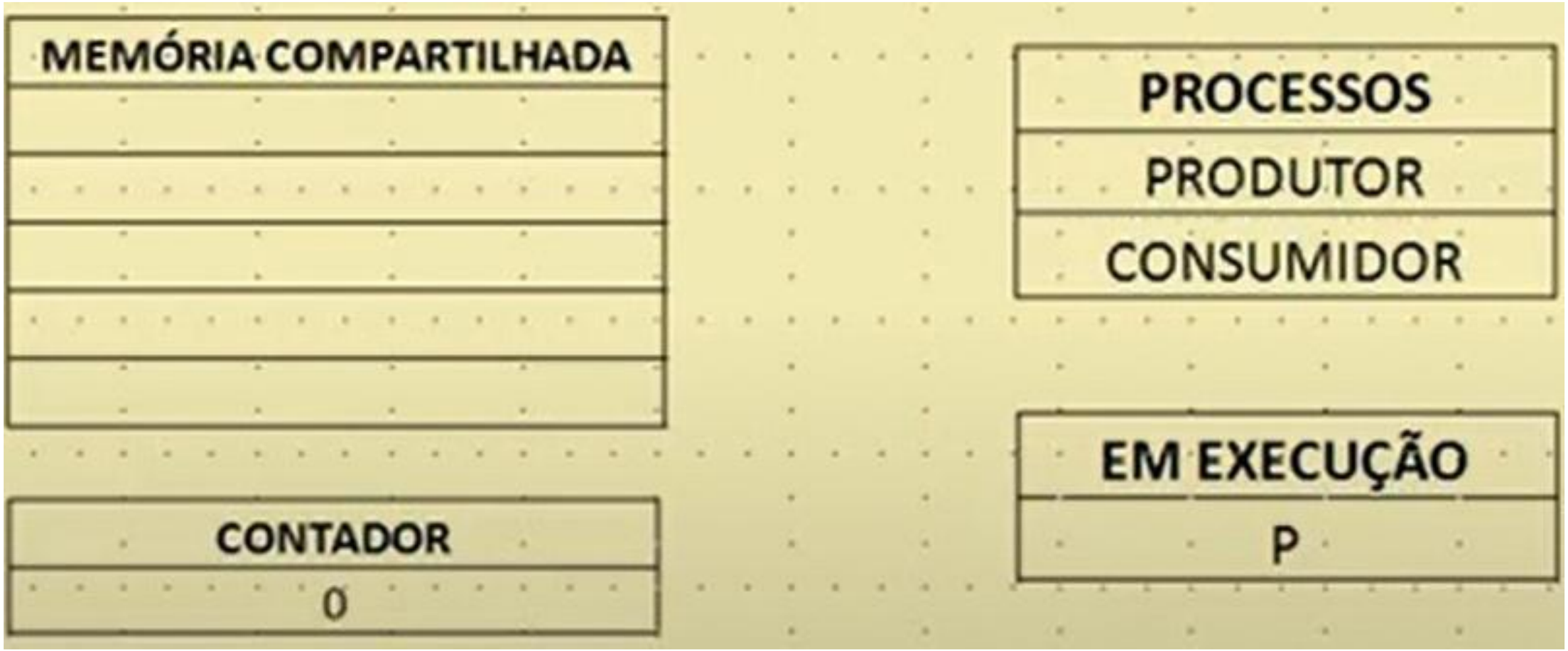
4. Processa Item: Executa o consumo final do dado armazenado em **next_consumed**.

💡 Exemplo Prático:

Como um cliente pegando pratos de uma esteira: se a esteira está vazia (counter == 0), ele precisa esperar até que o cozinheiro coloque um novo prato.



Antecedentes





Antecedentes: O Problema da Concorrência

⚠️ Concorrência Inesperada

1. Corretude Isolada:

As rotinas do **produtor** e do **consumidor** estão perfeitas quando executadas de forma individual.

2. Falha de Execução:

Quando executadas **concorrentemente** (ao mesmo tempo), o resultado final pode ser inconsistente e incorreto!

💡 **Ponto-Chave:**

A ordem exata de intercalação das instruções afeta diretamente o resultado final.

📊 Cenário Prático Exemplo

Estado Inicial Compartilhado:

counter = 5

Operações Simultâneas:

🟢 Produtor executa: `counter++`

🔴 Consumidor executa: `counter--`

Intuição inicial esperada:

5 + 1 - 1 = 5 (Esperado)

⚡ Resultado Inesperado

Após a execução concorrente:

O valor de ***counter*** pode ser:

4, 5 ou 6!

Por que isso acontece?

Comandos como `counter++` não são atômicos no nível de máquina (CPU); eles se desdobram em múltiplas instruções de registradores.



Antecedentes: A Não-Atomicidade do Comando

1. Visão de Alto Nível

O comando do programa:
`counter++`

⚠️ **Ilusão de Simplicidade:** Parece uma instrução única e indivisível (atômica), mas **não é**.

💡 **Risco Concorrente:** A CPU pode interromper a execução no meio desses passos para alternar de processo.

2. Linguagem de Máquina

Desdobramento em 3 passos:

1️⃣ **Leitura (RAM → CPU):**
```
registrador1 = counter
```

2️⃣ **Modificação (CPU):**
```
registrador1 =
registrador1 + 1
```

3️⃣ **Escrita (CPU → RAM):**
```
counter = registrador1
```

3. Arquitetura da CPU

💻 **Registrador Local:** `registrador1` é um registrador da CPU local.

🧠 **Memória Compartilhada:** A variável `counter` reside na memória principal acessada por múltiplos processos.

⚡ **Conseqüência:** Cada processo possui seus próprios registradores locais. A troca de contexto durante os passos 1, 2 ou 3 causa corrupção de dados!



Antecedentes: A Não-Atomicidade do Comando counter--

1. Visão de Alto Nível

O comando do programa:
`counter--`

⚠️ **Operação de Decremento:**
Reduz em 1 unidade a variável compartilhada, mas **não é atômica**.

💡 **Risco Concorrente:** A CPU pode interromper a execução no meio destas instruções para alternar para outro processo.

2. Linguagem de Máquina

Desdobramento em 3 passos:

1️⃣ **Leitura (RAM → CPU):**
```
registrador2 = counter
```

2️⃣ **Modificação (CPU):**
```
registrador2 =
registrador2 - 1
```

3️⃣ **Escrita (CPU → RAM):**
```
counter = registrador2
```

3. Arquitetura da CPU

💻 **Registrador Local:**
`registrador2` é um registrador privado da CPU local para este processo.

🧠 **Memória Compartilhada:** A variável `counter` fica localizada na memória RAM global.

⚡ **Consequência:** Intercalações entre os passos 1, 2 e 3 geram sobrescrita indevida de dados na memória.



Antecedentes: Concorrência e Intercalação

1. Regra de Intercalação

🔄 **Execução Simultânea:** Quando os comandos `counter++` e `counter--` rodam juntos, a CPU intercala suas instruções.

🔀 **Ordem Arbitrária:** O escalonador da CPU pode alternar entre os processos a qualquer momento.

📌 **Preservação de Ordem Interna:** A sequência individual dos 3 passos de cada comando NUNCA é invertida internamente.

💡 **Resultado: As 6 instruções de máquina (3 do ++ e 3 do --) misturam-se no tempo em sequências imprevisíveis!**

2. Exemplo Prático de Intercalação

Sequência temporal possível das instruções:

T0 [Produtor]: `reg1 = counter` (Lê 5)

T1 [Produtor]: `reg1 = reg1 + 1` (reg1 = 6)

T2 [Consumidor]: `reg2 = counter` (Lê 5)

T3 [Consumidor]: `reg2 = reg2 - 1` (reg2 = 4)

T4 [Produtor]: `counter = reg1` (Escreve 6)

T5 [Consumidor]: `counter = reg2` (Escreve 4!)



Antecedentes: Inconsistência do Estado do Buffer

1. Resultado Incorreto Obtido (counter == 4)

❌ **Erro Detectado:** O sistema indica **counter == 4** (4 buffers cheios), mas na realidade existem **5 buffers cheios**.

🔍 **Causa na Intercalação:** A instrução T5 do Consumidor (`counter = reg2`) foi executada por último.

💡 **Consequência:** O valor atualizado pelo Produtor em T4 (`counter = 6`) foi totalmente sobrescrito e perdido!

2. Inversão dos Comandos (T4 ↔ T5)

🔄 **E se invertêssemos T4 e T5?** Se a escrita de T4 (`counter = reg1`) ocorresse APÓS T5...

⚠️ **Novo Estado Incorreto:** Chegaríamos ao valor final **counter == 6** (também errado!).

📌 **Lição Chave:** O resultado final depende exclusivamente de quem escreve por último na memória RAM global.



Antecedentes: O Conceito de Condição de Corrida

1. Origem do Estado Incorreto

🔄 **Acesso Concorrente:** Permitimos que dois processos manipulassem a mesma variável `counter` simultaneamente.

⚙️ **Falta de Isolamento:** Sem restrição, operações de leitura e escrita se entrelaçam no tempo sem sincronismo.

💥 **Falha Resultante:** O valor final do dado fica totalmente corrompido em relação ao valor real esperado.

2. Condição de Corrida (Race Condition)

🏁 **Definição Formal:** Ocorre quando múltiplos processos acessam e modificam os mesmos dados concorrentemente.

⏱️ **Sensibilidade à Ordem:** O resultado final da execução depende diretamente da **ordem exata** em que os acessos ocorrem.

💡 **Exemplo Prático:** Como dois compradores clicando no último ingresso no mesmo milissegundo: quem gravar por último define o dono!



Antecedentes: Necessidade de Sincronização

1. Proteção da Variável counter

🎯 **Objetivo Principal:** Impedir o acesso simultâneo. Apenas **um processo por vez** pode alterar a variável `counter`.

⚠️ **O Risco sem Controle:** Execuções concorrentes sem regras causam a **condição de corrida**, corrompendo o valor final.

💡 **Exemplo Prático:** Pense em duas pessoas editando a mesma célula de uma planilha ao mesmo tempo: se não houver trava, uma sobrescreve a outra!

2. Mecanismo de Sincronização

🔒 **Exclusão Mútua:** Se o Processo A está alterando o dado, o Processo B **deve aguardar** até que A termine.

⏱️ **Coordenação de Tempo:** Os processos precisam de protocolos de comunicação e espera (como travas e sinais).

📌 **Lição Fundamental:** Sincronizar não é parar o sistema, mas sim garantir a **ordem correta de acesso** à memória compartilhada!



Exercício: Identificação de Inconsistência

1. Contexto da Execução

⚙️ **Cenário Sem Sincronização:**
Três processos (**P1**, **P2** e **P3**) executam a instrução `counter++` simultaneamente.

📌 **Estado Inicial:**
O valor inicial da variável na memória RAM é `counter = 0`.

🔍 **Etapas de Baixo Nível (`counter++`):**

- **Read:** Leitura do valor atual da RAM para o registrador do processo.
- **Increment:** Soma de +1 no valor armazenado no registrador.
- **Write:** Gravação do novo valor de volta na RAM.

2. Atividades do Exercício

✏️ **Com base na intercalação das operações, determine:**

- **Identificação de Sequências:**
  Descreva **três possíveis sequências** de execução (ordem de leituras e escritas) entre os processos P1, P2 e P3.
- **Valores Finais da Variável:**
  Calcule o **valor final de counter** para cada uma das 3 sequências identificadas.
- **Análise de Sobrescrita (Inconsistência):**
  Para cada sequência, identifique qual(is) processo(s) teve(ram) sua atualização sobrescrita na memória RAM.

Parte II - O problema da Seção Crítica



O Problema da Seção Crítica

1. Conceito e Definição

💻 **Ambiente Multi-processos:**
Considere um sistema com **n** processos concorrentes: `{P0, P1, ..., Pn-1}`.

🔍 **O que é Seção Crítica?**
É o trecho de código onde o processo manipula **recursos compartilhados**.

💡 **Exemplos Práticos de Operações:**

- Alterar variáveis globais (ex: `saldo += 100`)
- Atualizar registros em uma tabela do banco
- Escrever ou modificar dados em um arquivo comum

2. Regra de Ouro: Exclusão Mútua

🛡️ **A Regra Fundamental:**
Quando um processo está executando em sua seção crítica, **NENHUM outro processo** pode ter permissão para entrar na sua.

⚠️ **Por que isso é vital?**
A execução simultânea causa **corrupção de dados** e inconsistência no sistema.

🎯 **Analogia Simples (Caixa Eletrônico):**

Apenas uma pessoa pode usar o caixa eletrônico por vez. Outras pessoas devem aguardar na fila até que a sessão atual seja finalizada com segurança.

Estrutura Geral de um Processo Típico

💻 Fluxo do Algoritmo (do ... while)

```
do {
```

1. Seção de Entrada (entry section)
🔑 Solicita permissão para entrar

2. Seção Crítica (critical section)
⚠️ **Acesso exclusivo ao recurso!**

3. Seção de Saída (exit section)
🔓 Libera o recurso para os outros

4. Seção Remanescente (remainder)
⚙️ Execução de código local

```
} while (true);
```

📘 Entendendo Cada Etapa na Prática

1. Seção de Entrada (Entry Section)

Solicita a trava (lock). O processo aguarda na fila até receber autorização para prosseguir. Exemplo: `adquirir_trava()`.

2. Seção Crítica (Critical Section)

Momento em que o processo altera variáveis globais, arquivos ou tabelas do banco. **Apenas 1 processo por vez!** Exemplo: `saldo += 100`.

3. Seção de Saída (Exit Section)

Sinaliza que terminou de usar o recurso, permitindo que o próximo processo da fila entre. Exemplo: `liberar_trava()`.

4. Seção Remanescente (Remainder Section)

Qualquer outra tarefa local do programa que não afeta recursos compartilhados (ex: processamento de telas, logs locais).

Requisitos para Solução da Seção Crítica (1)

Uma solução válida deve satisfazer obrigatoriamente a **3 requisitos vitais** de sincronização:

1. Exclusão Mútua (Mutual Exclusion)

🛡️ **Regra Fundamental:**
Se o processo **Pi** está executando em sua seção crítica, **nenhum outro processo** pode executar a sua ao mesmo tempo.

⚠️ **Por que é indispensável?**
Evita condições de corrida (race conditions) e corrupção nos dados compartilhados.

🔒 **Acesso Exclusivo:**
Garante que a manipulação de recursos compartilhados seja estritamente isolada e segura.

💡 Entendendo na Prática

🔑 **Exemplo da Cabine Individual:**
Pense em um provador de roupas trancado por dentro. Enquanto uma pessoa (**Pi**) está lá dentro, ninguém mais entra até que ela destranque e saia.

🚪 **A Trava (Lock):**
O fecho da porta funciona como a trava do sistema. Quem possui o acesso impede qualquer outro de entrar.

🎯 **Resumo para Fixar:**
Apenas **uma pessoa por vez** no provador; todos os outros aguardam do lado de fora!

Requisitos para Solução da Seção Crítica (2)

Uma solução válida deve satisfazer obrigatoriamente a **3 requisitos vitais** de sincronização:

2. Espera Limitada (Bounded Waiting)

⏳ **Regra Fundamental:**
Existe um **limite (fronteira)** no número de vezes que outros processos entram na seção crítica após **Pi** solicitar acesso.

⚠️ **Por que é indispensável?**
Garante que o atendimento não seja adiado indefinidamente, impedindo a **Inanição (Starvation)**.

🛑 **Fronteira Definida:**
Estabelece um teto máximo de espera para que nenhum processo aguarde para sempre.

💡 Entendendo na Prática

🎟️ **Exemplo da Fila com Senha:**
Ao pegar a senha #15 em um banco, no máximo 14 pessoas serão atendidas antes de você.

🎫 **A Fila Organizada:**
A senha estabelece um limite claro: outros processos não podem entrar infinitas vezes na sua frente.

🎯 **Resumo para Fixar:**
Sua vez **sempre vai chegar**! Ninguém fura a fila infinitamente à sua frente.

Requisitos para Solução da Seção Crítica (3)

Uma solução válida deve satisfazer obrigatoriamente a **3 requisitos vitais** de sincronização:

3. Progresso (Progress)

⚡ **Regra Fundamental:**
Se a seção crítica está livre e existem processos querendo entrar, **somente** aqueles fora da seção remanescente participam da decisão.

⚠️ **Por que é indispensável?**
Impede travamentos do sistema (**Deadlocks**) e garante que o sistema continue progredindo.

⌛ **Sem Adiamento Indefinido:**
A seleção de qual processo entrará a seguir **não pode ser postergada** para sempre.

💡 Entendendo na Prática

🚪 **Exemplo da Sala de Reunião:**
Imagine uma sala vaga. Três pessoas querem usá-la agora. Apenas essas 3 pessoas decidem quem entra primeiro.

🚫 **O que NÃO pode acontecer:**
Uma quarta pessoa que está em casa descansando (seção remanescente) **não pode opinar** nem impedir a entrada das outras.

🎯 **Resumo para Fixar:**
Quem não precisa do recurso compartilhado **não pode atrapalhar** quem está precisando dele!

O Problema: Condição de Corrida no Kernel

Como a falta de sincronização em estruturas compartilhadas gera inconsistências no sistema:

1. Estrutura de Dados do Kernel

📁 **Lista Global de Arquivos:**
O kernel mantém uma tabela centralizada contendo todos os arquivos abertos no sistema.

🔄 **Modificações Constantes:**
A lista deve ser alterada sempre que um arquivo é **aberto** (adicionando elemento) ou **fechado** (removendo elemento).

👥 **Acesso Concorrente:**
Múltiplos processos solicitam abertura e fechamento de arquivos ao mesmo tempo.

2. A Condição de Corrida

⚡ **Concorrência sem Proteção:**
Ocorre quando dois processos tentam atualizar a mesma lista simultaneamente.

💡 **Exemplo Prático:**
Se o **Processo A** e o **Processo B** abrem arquivos no mesmo instante, o Processo B pode sobrescrever o ponteiro do Processo A antes da gravação terminar.

⚠️ **Consequência Grave:**
A lista fica corrompida, gerando perda de ponteiros, inconsistências ou falha do sistema (Kernel Panic).

O Problema: Kernel com Preempção

Como o sistema operacional lida com interrupções em modo kernel para garantir responsividade:

1. O Conceito de Preempção

⚡ **Interrupção Permitida:**
Um processo executando no modo kernel **pode ser interrompido** a qualquer instante por outro de maior prioridade.

🔄 **Alta Responsividade:**
Evita que tarefas demoradas do sistema bloqueiem ações urgentes e da interface com o usuário.

⚠️ **Desafio de Sincronização:**
Exige cuidado rigoroso com **condições de corrida** ao acessar dados compartilhados do kernel.

💡 Entendendo na Prática

🏥 **Exemplo da Pronta-Emergência:**
Um médico (CPU) atende um caso de rotina. Se chega um caso grave (emergência), a consulta é **pausada imediatamente** para atender a prioridade.

🎯 **Vantagem Principal:**
Processos em tempo real e interações do usuário **não ficam congelados** esperando rotinas longas terminarem.

🔒 **Resumo para Fixar:**
Preempção garante **agilidade e prioridade**, mas exige travas de segurança na memória do kernel!

O Problema: Kernel sem Preempção



Como o conceito de atomicidade sem interrupção afeta o desempenho e a responsividade:

1. Atomicidade ("Tudo ou Nada")

⚛️ **Execução Ininterrupta:**
O processo deve ser executado por completo ou não ser executado. Nenhuma pausa é permitida no meio.

🛑 **Uso em Kernels Sem Preempção:**
O processo assume o controle da CPU no modo kernel e não abre mão dele até terminar sua tarefa.

⚠️ **O Grande Gargalo:**
Processos muito longos tomam a CPU e **bloqueiam todos os outros processos** do sistema.

💡 Entendendo na Prática

🛒 **Exemplo do Caixa Único:**
Imagine um único caixa no mercado atendedo um cliente com **10.000 itens**. Ninguém mais passa até ele passar o último item!

⏳ **Impacto no Sistema:**
Se uma rotina de disco demorar 10 segundos, todo o sistema operacional e a tela **ficam "congelados"** esperando.

🔒 **Resumo para Fixar:**
Atomicidade garante **segurança de dados**, mas sem preempção traz **pessima responsividade**!



Duvidas?

Parte III - Solução de Peterson



Solucionando o Problema da Seção Crítica

Dentre as formas de solucionar o problema da Seção Crítica, temos a **Solução de Peterson**.

⚡ Solução Simples

Algoritmo clássico baseado em software para garantir exclusão mútua sem recursos de hardware complexos.

👥 Apenas Dois Processos

Projetado estritamente para alternância segura entre dois processos (P0 e P1).

⏳ Espera Ocupada!

Usa laço ativo (while) aguardando liberação da Seção Crítica, consumindo CPU (Spinlock).

Exemplo Prático: Estrutura da Solução de Peterson

```
int turn;
boolean flag[2];
```

Processo Pi (ex: P0)

```
do {
 flag[i] = true;
 turn = j;
 while (flag[j] && turn == j);
        🔒 SEÇÃO CRÍTICA
 flag[i] = false;
 Seção Remanescente;
} while (true);
```

Recurso Compartilhado

Apenas **um processo** pode executar sua Seção Crítica por vez.



Solucionando o Problema da Seção Crítica

A solução de Peterson requer que os dois processos compartilhem dois itens de dados:

`int turn;` Indica a vez de acesso

Define qual processo está autorizado a entrar em sua seção crítica.

`turn == 0 (Vez do P0)` | **`turn == 1 (Vez do P1)`**

`boolean flag[2];` Sinaliza intenção

Indica se um processo está pronto/deseja entrar na seção crítica.

flag[0] = true
P0 pronto para entrar
flag[1] = true
P1 pronto para entrar

💡 Exemplo de Arquitetura de Memória Compartilhada

⚙️ Processo P0
Executa código de entrada e define dados compartilhados.

⚙️ Processo P1
Executa código de entrada e verifica a vez do concorrente.

📦 Memória Compartilhada

```
int turn;
boolean flag[2];
```

🔒 SEÇÃO CRÍTICA
Apenas 1 processo por vez pode acessar!



Solucionando o Problema da Seção Crítica

Mecanismo de Entrada na Seção Crítica pelo Processo Pi

🚩 1. Sinaliza Intenção

`flag[i] = true;`

Primeiro, o processo **Pi** posiciona seu indicador como **true** para anunciar que deseja entrar na seção crítica.

🔄 2. Cede a Vez (Cortesia)

`turn = j;`

Em seguida, atribui a **turn** o valor **j**, assegurando que se o outro processo quiser entrar, ele possa fazê-lo.

💻 Algoritmo da Solução de Peterson (Processo Pi)

```
do {
  flag[i] = true;  ← Passo 1: Quero entrar
  turn = j;  ← Passo 2: Sua vez, Pj
  while (flag[j] && turn == j);  ← Espera ocupada
                  🔒 SEÇÃO CRÍTICA
  flag[i] = false;  ← Libera a seção
  Seção Remanescente;
} while (true);
```



Solucionando o Problema da Seção Crítica

Simultaneidade na alteração de variáveis compartilhadas e resolução de conflito

⚙️ Processo P0 (i)

Executa simultaneamente:

```
flag[0] = true;
turn = 1;
```

(Atribui vez a P1)

Tenta escrever na memória compartilhada ao mesmo tempo que P1.

📦 Memória Compartilhada (turn)

⚡ Disputa de Escrita Quase Simultânea

```
int turn;
```

Última escrita prevalece (Sobreposição)

Apenas **uma atribuição** permanece gravada na memória de forma definitiva.

⚙️ Processo P1 (j)

Executa simultaneamente:

```
flag[1] = true;
turn = 0;
```

(Atribui vez a P0)

Tenta escrever na memória compartilhada ao mesmo tempo que P0.

⏱️ 1. Atribuição Simultânea

Se os dois processos tentarem entrar ao mesmo tempo, a variável `turn` será posicionada tanto com **i** quanto com **j** quase no mesmo instante.

🔄 2. Sobreposição Imediata

Apenas uma dessas atribuições permanecerá na memória. A outra escrita ocorrerá, mas será sobreposta imediatamente pela escrita mais recente.

🔑 3. Decisão do Acesso

O valor final de `turn` determina qual dos dois processos é autorizado a entrar primeiro em sua seção crítica.



Solucionando o Problema da Seção Crítica

Provamos, agora, que essa solução está correta. Precisamos mostrar que:

🛡️ Exclusão Mútua

1. A exclusão mútua é preservada.

🔒

`P0 em SC ➔ P1 bloqueado`

Apenas **um único processo** pode executar em sua seção crítica por vez.

Se *P0* está acessando a memória compartilhada, *P1* é estritamente impedido de entrar.

⚡ Progresso

2. O requisito de progresso é atendido.

🔄

`Sem Deadlock / Atraso`

Se nenhum processo está na seção crítica e alguns desejam entrar, a decisão não pode ser adiada indefinidamente.

Apenas os processos que não estão na seção remanescente participam da escolha.

⏳ Espera Limitada

3. O requisito de espera limitada é atendido.

⏱️

`Sem Starvation`

Existe um limite fixo de vezes que outros processos entram na seção crítica após um pedido ter sido feito.

Garante que nenhum processo espere para sempre (previne fome de processos).



Solucionando o Problema da Seção Crítica

Demonstração Visual e Análise do Algoritmo de Peterson

🔍 1. Condição de Entrada de Pi

O processo **Pi** só entra na Seção Crítica se pelo menos uma destas condições for verdadeira:

`flag[j] == false` **OU** `turn == i`

⚡ 2. Hipótese de Concorrência Simultânea

Se **ambos os processos** pudessem executar suas seções críticas ao mesmo tempo:

`flag[0] == flag[1] == true`

🛡️ Conclusão da Exclusão Mútua

Como a variável `turn` só pode assumir **um único valor (0 ou 1)** por vez, apenas um processo consegue avançar. A exclusão mútua é estritamente garantida!

💻 Estrutura do Código (Processo P_i)

```
do {
flag[i] = true;
turn = j;
while (flag[j] && turn == j);
                    🔒 [ SEÇÃO CRÍTICA ]
flag[i] = false;
                    ⚙️ Seção Remanescente
} while (true);
```



Solucionando o Problema da Seção Crítica

Prova das Propriedades 2 e 3 (Progresso e Espera Limitada)

🔍 Análise de Bloqueio

Para provar as **propriedades 2** e **3**, observamos o comportamento do processo ***Pi***.

O processo ***Pi*** só pode ser impedido de entrar na seção crítica se ficar preso na espera ocupativa.

⚡ Condição do Loop

O bloqueio ocorre **somente** no loop `while` com a condição:

`flag[j] == true && turn == j`

Esse loop é o único ponto de espera possível no algoritmo!

💻 Algoritmo de Peterson (Processo P_i)

```
do {
  flag[i] = true;
  turn = j;
// Ponto de Bloqueio Único
while (flag[j] && turn == j);
                🔒 [ SEÇÃO CRÍTICA ]
  flag[i] = false;
                ⚙️ Seção Remanescente
} while (true);
```

Solucionando o Problema da Seção Crítica

Análise dos Casos de Execução do Algoritmo de Peterson

🔍 Caso 1: Pj Não Pronto (flag[j] == false)

Se **Pj** não estiver pronto para entrar na seção crítica, então `flag[j] == false`.

➡️ **Pi pode entrar livremente em sua seção crítica**

⚡ Caso 2: Pj em Execução do Loop (flag[j] == true)

Se **Pj** posicionou `flag[j]` como `true` e também está executando seu comando **`while`**:

`turn == i OU turn == j`

Apenas o processo cujo número é igual a turn entrará na seção crítica!

💻 Algoritmo de Peterson (Processo P_i)

```
do {
  flag[i] = true;
  turn = j;
// Ponto de Validação da Seção Crítica
while (flag[j] && turn == j);
                🔒 [ SEÇÃO CRÍTICA ]
  flag[i] = false;
                ⚙️ Seção Remanescente
} while (true);
```

Solucionando o Problema da Seção Crítica

Funcionamento e Troca de Turno no Algoritmo

🔑 Regra de Entrada por Turno

Se `turn == i`, então o processo ***Pi*** entrará na seção crítica.

Se `turn == j`, então o processo ***Pj*** entrará na seção crítica.

🔄 **A variável turn define a prioridade de entrada**

🔓 Liberação e Saída da Seção Crítica

Quando ***Pj*** sair de sua seção crítica, ele posicionará a flag:

`flag[j] = false;`

Isso encerra o loop de espera, permitindo que ***Pi*** finalmente entre em sua seção crítica com segurança.

💻 Algoritmo de Peterson (Processo P_i)

```
do {
  flag[i] = true;
  turn = j;
// Ponto de Validação da Seção Crítica
while (flag[j] && turn == j);
                🔒 [ SEÇÃO CRÍTICA ]
  flag[i] = false;
                ⚙️ Seção Remanescente
} while (true);
```

Solucionando o Problema da Seção Crítica

Garantia de Progresso e Espera Limitada no Algoritmo de Peterson

⏳ Garantia de Progresso (Execução de Pj)

Se **Pj** posicionar `flag[j] = true`, ele também deve atribuir a `turn` o valor **i**.

Como **Pi** não altera `turn` no loop **`while`**, **Pi** entrará na seção crítica!

➡️ **A variável turn garante que não haverá adiamento indefinido**

🛡️ Espera Limitada (Bounded Waiting)

O processo **Pi** entrará na seção crítica (progresso) após no máximo **uma única entrada** de **Pj**.

`Espera Limitada Garantida`

✅ **Evita Starvation: Nenhum processo espera indefinidamente por acesso.**

💻 Algoritmo de Peterson (Processo P_i)

```
do {
  flag[i] = true;
  turn = j;
// Validação de Progresso e Espera Limitada
while (flag[j] && turn == j);
                🔒 [ SEÇÃO CRÍTICA ]
  flag[i] = false;
                ⚙️ Seção Remanescente
} while (true);
```

Desvantagens da Solução de Peterson

Principais Limitações de Desempenho e Escalabilidade

⏳ 1. Espera Ocupada (Busy Waiting)

⚠️ O Problema do Desperdício de CPU:

O processo permanece executando continuamente um laço de repetição enquanto aguarda a sua vez de entrar na seção crítica.

```
// Exemplo Prático de Código:
while (flag[j] && turn == j);
```

🔁 CPU roda a 100% testando esta condição sem parar!

📉 Impactos no Sistema:

- Consumo inútil de processamento e energia.
- Prejudica outros processos que poderiam usar a CPU.
- Inviável em sistemas com baterias ou alta carga.

👥 2. Limitação a Dois Processos

🚫 Falta de Escalabilidade:

A lógica do algoritmo foi projetada estritamente para alternar o acesso entre apenas 2 processos (P_0 e P_1).

✅ **Suportado: Processo P0 ↔ Processo P1**

❌ **Não suportado: P0, P1, P2, P3 ... Pn**

⚙️ Por que não funciona em N processos?

- Usa apenas um vetor `flag[2]` e a variável `turn`.
- Para concorrer com mais processos, exige extensões complexas (como a Padaria de Lamport).
- Sistemas operacionais modernos exigem suporte nativo a múltiplos threads/processos.

Sugestão de vídeo!!!

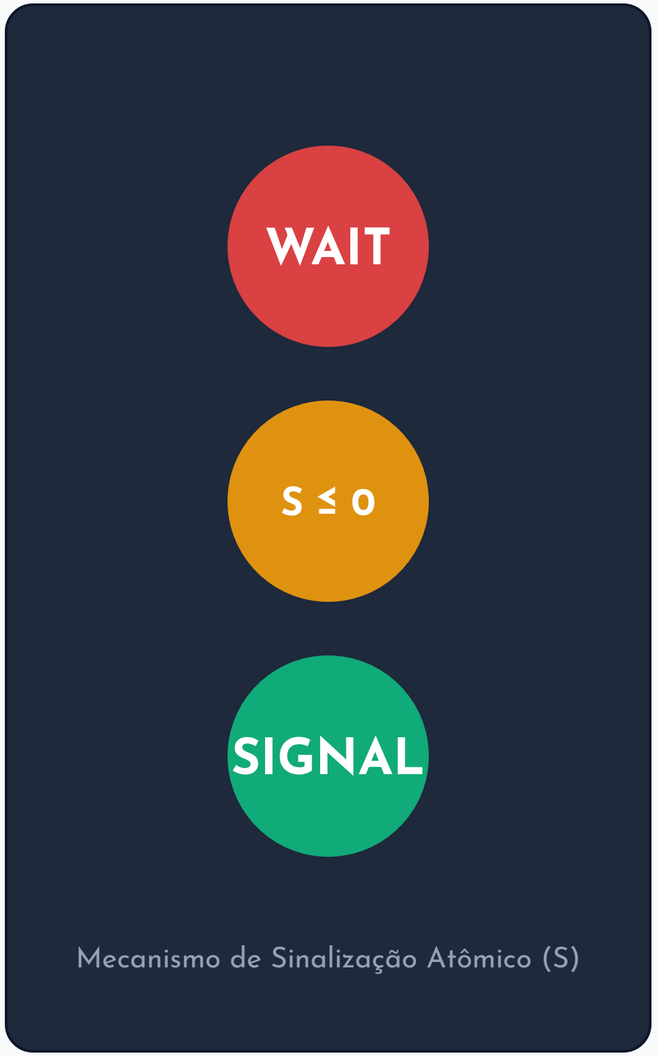


https://www.youtube.com/watch?v=_lh0FRox0OY

Parte IV - Semáforos

Parte IV - Semáforos

O que são e vantagens em relação à Solução de Peterson



Mecanismo de Sinalização Atômico (S)

⚡ 1. Fim da Espera Ocupada (Busy Waiting)

Solução de Peterson: Mantém a CPU em um laço while() a 100% gastando energia.

Semáforos: Bloqueiam o processo na fila de espera do SO (dorme), liberando a CPU para outras tarefas.

🛠️ 3. Flexibilidade na Gestão de Recursos

Semáforo Binario (Mutex): Garante exclusão mútua simples (0 ou 1).

Semáforo Contador: Permite controlar o acesso a S cópias idênticas de um recurso compartilhado.



Soluções com Semáforos

Um **semáforo S** é uma **variável inteira** que, exceto na inicialização, é acessada apenas por meio de duas operações atômicas padrão: ***wait()*** e ***signal()***.

Exemplo Prático de Funcionamento (Controle de Recurso Compartilhado)

Operação wait(S)

Decrementar / Aguardar

Solicita acesso ao recurso compartilhado.

```
wait(S) {
  while (S <= 0);
  S--;
}
```

- Se S > 0: decrementa e entra.
- Se S ≤ 0: bloqueado e aguarda em espera (fila de espera).

Estado do Semáforo (S)

S = Valor Inteiro

Controla a quantidade de processos autorizados a acessar a Seção Crítica simultaneamente.

Operação signal(S)

Incrementar / Liberar

Libera o recurso após a utilização na Seção Crítica.

```
signal(S) {
  S++;
}
```

- Incrementa S.
- Notifica outros processos em espera.

Soluções com Semáforos

Garantia de Indivisibilidade e Operações Atômicas

1. Modificação Indivisível (Atômica)

Regra: As alterações no valor do semáforo em ***wait()*** e ***signal()*** são estritamente exclusivas.

💡 **Exemplo Prático:**

Se o **Processo A** está modificando o semáforo *S*, nenhum outro processo pode alterar *S* ao mesmo tempo.

Cenário de Execução:

➔ **Processo A: Modificando S...**

🚫 **Processo B: BLOQUEADO**

Deve aguardar o término da operação do Processo A.

Garante que a variável *S* não seja corrompida por acessos simultâneos.

2. Teste e Decremento sem Interrupção

Regra: No ***wait(S)***, a verificação do valor **(S ≤ 0)** e a alteração **(S--)** ocorrem de forma contínua.

💡 **Exemplo de Problema Evitado:**

Evita que dois processos vejam *S = 1* simultaneamente e ambos consigam decrementar e entrar.

Bloco Atômico do wait(S):

```
wait(S) {
  while (S <= 0); // Teste
  S--;            // Decremento
}
```

✔ **Operação Executada Sem Interrupção do Hardware**



Uso dos Semáforos de Contagem

Gerenciamento de Acesso a Recursos com Instâncias Finitas

Regras de Funcionamento

1. Inicialização:
O semáforo é iniciado com o **número total de recursos disponíveis (N)**.

2. Operação wait() [Solicitar]:
Cada processo que precisa de um recurso executa ***wait()***.

- Decrementa a contagem (**S--**)
- Se **S < 0**, o processo fica **bloqueado** até um recurso ser liberado.

3. Operação signal() [Liberar]:
Ao finalizar o uso, o processo executa ***signal()***.

- Incrementa a contagem (**S++**)
- Libera o próximo processo que estava aguardando na fila.

Exemplo Prático: Controle de Vagas (N = 2)

Estado Inicial: Semáforo S = 2 (2 Instâncias Livres)

➔ Passagem 1: Processo P1 executa wait()
Contagem: **S = 1** | Recurso concedido a P1 imediatamente.

➔ Passagem 2: Processo P2 executa wait()
Contagem: **S = 0** | Recurso concedido a P2 (recursos esgotados).

➔ Passagem 3: Processo P3 executa wait()
Contagem: **S = -1** | P3 fica **BLOQUEADO** na fila de espera!

➔ Passagem 4: Processo P1 conclui e executa signal()
Contagem: **S = 0** | P1 libera a vaga, P3 acorda e assume o recurso!

Uso dos Semáforos: Sincronização

Garantia da Ordem de Execução entre Processos Concorrentes

1. O Problema da Sincronização

Cenário Concorrente:

Dois processos executam ao mesmo tempo: ***P1*** (com comando ***S1***) e ***P2*** (com comando ***S2***).

🎯 **Requisito Obrigatório:**

A instrução ***S2*** só pode ser executada **APÓS** a conclusão total de ***S1***.

📊 **Diagrama do Fluxo Desejado:**

Processo P1: Executa **S1** ➔ Finaliza
⬇️ *(Sinal de Liberação)*
Processo P2: Aguarda ➔ Executa **S2**

2. Solução com Semáforo Compartilhado

⚙️ **Mecanismo: synch = 0**

Ambos compartilham o semáforo ***synch***, inicializado com valor **0**.

💻 **Implementação dos Processos:**

```
// Processo P1:
S1;
signal(synch); // Libera P2 (synch++)

// Processo P2:
wait(synch); // Bloqueia até synch > 0
S2;
```

Por que funciona? Como synch = 0, se **P2** tentar executar primeiro, o wait() o bloqueia. Apenas quando **P1** conclui S1 e chama signal() o processo **P2** é desbloqueado para executar S2.

Implementação dos Semáforos

Evitando a Espera Ativa (Busy Waiting) com Bloqueio de Processos

1. Mecanismo de Bloqueio

1. Execução do wait():
Quando o processo executa ***wait()*** e o semáforo não é positivo, ele deve aguardar.

2. Fim da Espera Ativa:
Em vez de ficar em loop (**espera em ação**), o processo bloqueia a si próprio.

3. Fila do Semáforo:
- **Insere o processo na fila de espera do semáforo**
- **Muda o estado para Estado de Espera (Bloqueado)**

4. Liberação do Processador:
O controle vai para o **scheduler da CPU**, que escolhe outro processo.

2. Exemplo de Fluxo (Diagrama)

⚙️ **Processo P1 tenta acessar o recurso:**

Passo 1: P1 chama ***wait(S)*** ➔ Semáforo **S ≤ 0**

Passo 2: P1 executa **block()** [Auto-bloqueio]

Passo 3: P1 entra na **Fila do Semáforo** [[P1]]

Passo 4: **CPU Scheduler** seleciona Processo P2

💡 **Ganho de Eficiência:** A CPU não perde ciclos em loop e passa a executar outro trabalho útil!

Implementação dos Semáforos

Desbloqueio de Processos e Reativação pela CPU

1. Mecanismo de Desbloqueio

🔔 Disparo da Operação signal():

Quando um processo ativo executa ***signal(S)***, um processo que estava bloqueado esperando em **S** deve ser acordado.

⚡ Operação wakeup():

A chamada ***wakeup()*** é acionada, retirando o processo do estado de espera e alterando seu estado diretamente para **Pronto (Ready)**.

🔄 Transição de Estado:

[Estado: Bloqueado]
⬇️ **signal() ➔ wakeup()**
[Estado: Pronto]

2. Fila de Prontos e Escalonação

📥 Entrada na Fila de Prontos:

Após o **wakeup()**, o processo reiniciado é inserido imediatamente na **Fila de Prontos (Ready Queue)** para aguardar tempo de CPU.

🖥️ Papel do Escalonador da CPU:

A CPU pode ou não ser desviada do processo em execução para o novo processo pronto, dependendo do **algoritmo de scheduling** (preemptivo vs. não-preemptivo).

💡 Resumo do Fluxo:

Fila de Espera ➔ **wakeup()** ➔ Fila de Prontos ➔ **CPU Scheduling** ➔ Execução



Implementação do Semáforos

Para implementar semáforos usando essa definição, especificamos um semáforo como a estrutura mostrada a seguir:

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

Implementação dos Semáforos

Estrutura de Dados para Controle de Processos e Lista de Espera

1. Definição da Estrutura (C)

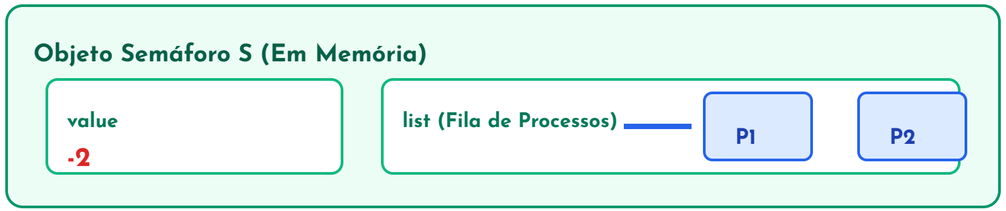
💻 Código da Estrutura em C:

```c
typedef struct {
  int value; // Contador
  struct process *list; // Fila
} semaphore;
```

📌 Componentes do Semáforo:

- 🔹 **value (Inteiro):** Armazena o saldo de recursos disponíveis.
- 🔹 ***list (Ponteiro):** Aponta para a lista encadeada de processos bloqueados aguardando o recurso.

2. Diagrama Visual e Exemplo



💡 Entendendo o Exemplo:

- Se **value < 0**, o seu valor absoluto (**|-2| = 2**) indica exatamente o número de processos bloqueados na lista.
- Os processos **P1** e **P2** tentaram executar ***wait()*** quando não havia recursos e foram colocados em estado de espera.

⚙️ Execução sem Espera Ativa:

A CPU não perde ciclos em loop! Os processos ficam suspensos na lista até receberem a chamada ***signal()***.

Implementação dos Semáforos

Operação wait( ) – Solicitando Recursos e Bloqueando Processos

1. Código da Operação wait( )

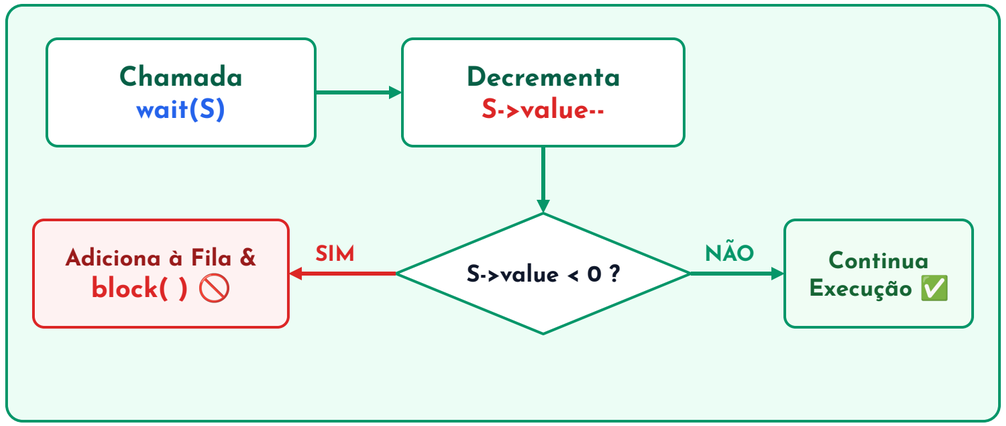
💻 Definição da Função em C:

```
wait(semaphore *S) {
  S->value--; // Requisita recurso
  if (S->value < 0) {
    adiciona este processo a S->list;
    block(); // Suspende o processo
  }
}
```

📌 Passo a Passo de Execução:

1. **S->value--**: O contador de recursos disponíveis é decrementado.
2. **Teste S->value < 0**: Se o valor for menor que zero, o recurso estava indisponível.
3. **Inclusão na Fila**: O processo atual é colocado na lista de espera **S->list**.
4. **Chamada block( )**: O SO suspende o processo até que um sinal chegue.

2. Fluxo Visual de Funcionamento



💡 Pontos Chave do Bloqueio:

- **Sem Espera Ativa (Busy Waiting)**: O processo não consome ciclos de CPU repetindo loops.
- **Estado Suspenso**: A instrução ***block()*** muda o estado do processo para bloqueado e transfere o controle ao escalonador.
- **Significado do Valor Negativo**: Se **value = -K**, existem exatamente **K** processos aguardando na fila.

Implementação dos Semáforos

Operação signal( ) – Liberando Recursos e Desbloqueando Processos

1. Código da Operação signal( )

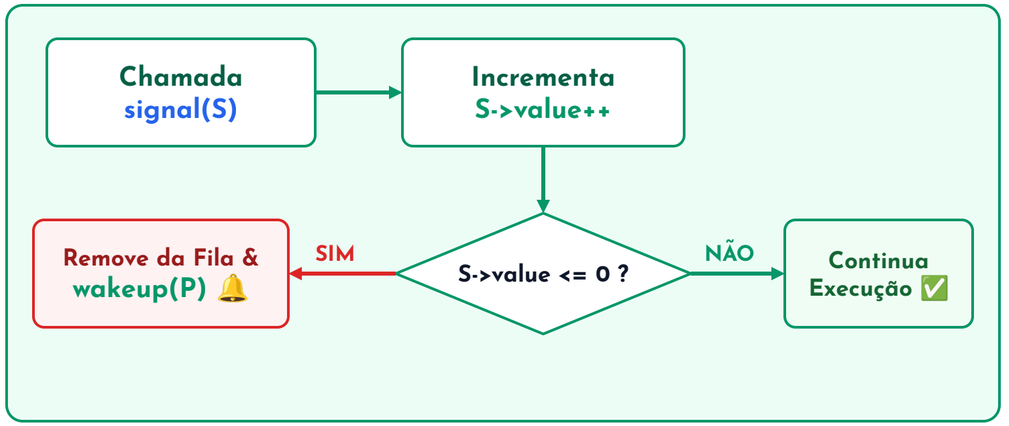
💻 Definição da Função em C:

```
signal(semaphore *S) {
  S->value++; // Libera recurso
  if (S->value <= 0) {
    remove um processo P de S->list;
    wakeup(P); // Acorda o processo
  }
}
```

📌 Passo a Passo de Execução:

1. **S->value++**: O contador de recursos disponíveis é incrementado.
2. **Teste S->value <= 0**: Se ainda for menor ou igual a zero, há processos na fila.
3. **Remoção da Fila**: Retira o primeiro processo **P** de **S->list**.
4. **Chamada wakeup(P)**: O SO reativa o processo P para pronto para executar.

2. Fluxo Visual de Funcionamento



💡 Pontos Chave da Liberação:

- **Notificação Ativa**: Se houver processos aguardando na fila, um é acordado imediatamente.
- **Estado Reativado**: A instrução ***wakeup(P)*** altera o estado do processo de bloqueado para pronto.
- **Significado do Valor Negativo**: Se mesmo após incrementar o valor continuar **<= 0**, ainda existem outros processos suspensos na fila.

Implementação dos Semáforos

Operações Básicas (block/wakeup) e Estrutura de Fila

1. Operações Básicas e Estrutura em C

💻 Exemplo de Estrutura do Semáforo:

```
typedef struct {
int value; // Valor inteiro
struct PCB *head; // Fila FIFO
struct PCB *tail;
} semaphore;
```

⚙️ Chamadas de Sistema do SO:

• **block( )**: Suspende o processo que a invocou e o coloca na fila do semáforo.

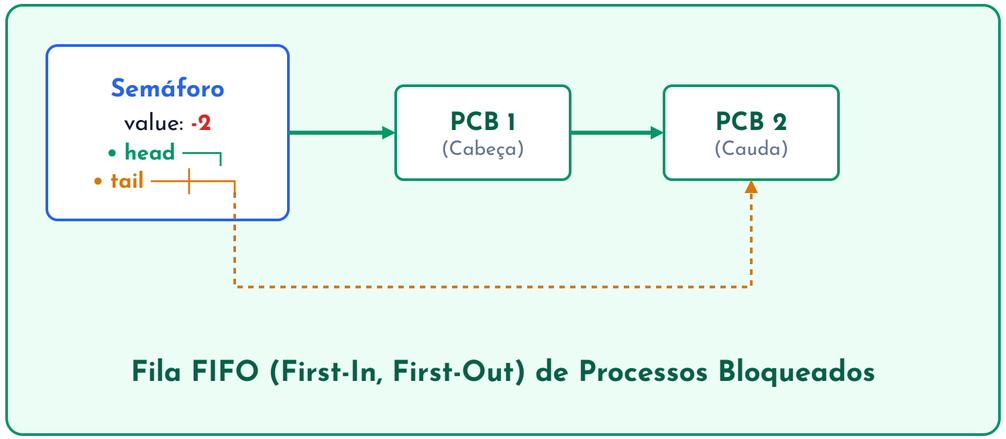
• **wakeup(P)**: Retoma a execução de um processo **P** que estava bloqueado.

📌 Componentes Essenciais:

• **Valor Inteiro**: Registra a quantidade de recursos ou o saldo de processos em espera.

• **Lista de PCBs**: Ponteiros de controle mantidos pelo SO.

2. Diagrama da Fila FIFO de PCBs



💡 Garantia de Espera Limitada:

• **Fila FIFO**: Adiciona novos processos no final e remove do início.

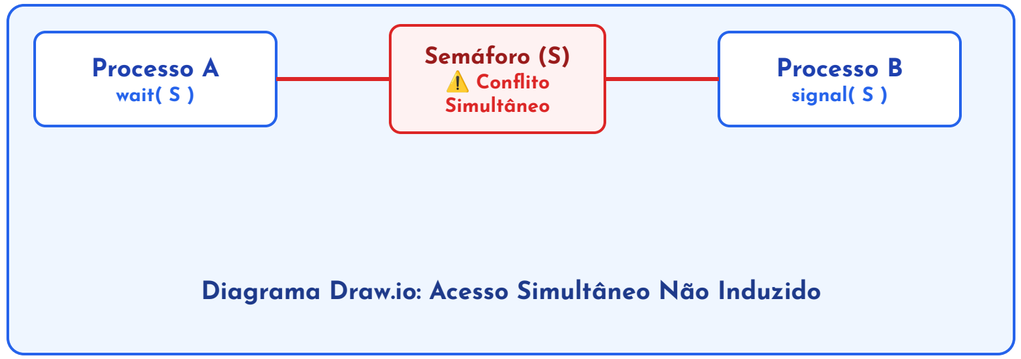
• **Sem Starvation**: Garante ordem justa de atendimento para todos os processos suspensos.

• **Ponteiros Cabeça/Cauda**: Acesso rápido em tempo constante O(1) para inserções e remoções.

Implementação dos Semáforos

Garantia de Atomicidade e Solução para Ambiente Uniprocessador

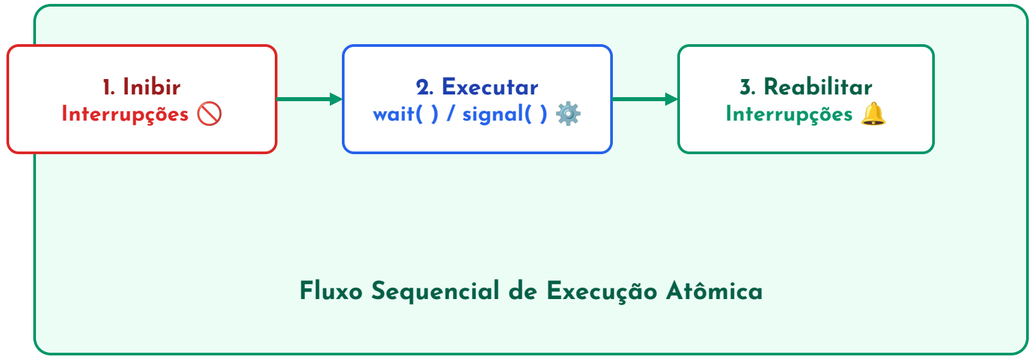
1. O Problema da Atomicidade



📌 Requisito Fundamental:

- **Execução Atômica:** As operações **wait()** e **signal()** não podem ser interrompidas.
- **Exclusão Mútua:** Impedir rigorosamente que dois processos modifiquem o mesmo semáforo ao mesmo tempo.
- **Risco de Inconsistência:** Intercalaçoes na alteração do valor levam a condições de corrida.

2. Solução em Uniprocessador



💡 Como Funciona na Prática:

- **Bloqueio de Troca de Contexto:** Desativar interrupções impede que o SO passe a CPU para outro processo.
- **Garantia Sequencial:** A instrução atual é finalizada do início ao fim sem trocas no meio.
- **Simplicidade Eficiente:** Solução ideal e completa para ambientes com **apenas uma CPU**.



Implementação do Semáforos

Em um ambiente multiprocessador, as interrupções devem ser desabilitadas em cada processador.

Caso contrário, instruções de processos diferentes (sendo executados em processadores diferentes) podem ser intercaladas de alguma forma arbitrária.



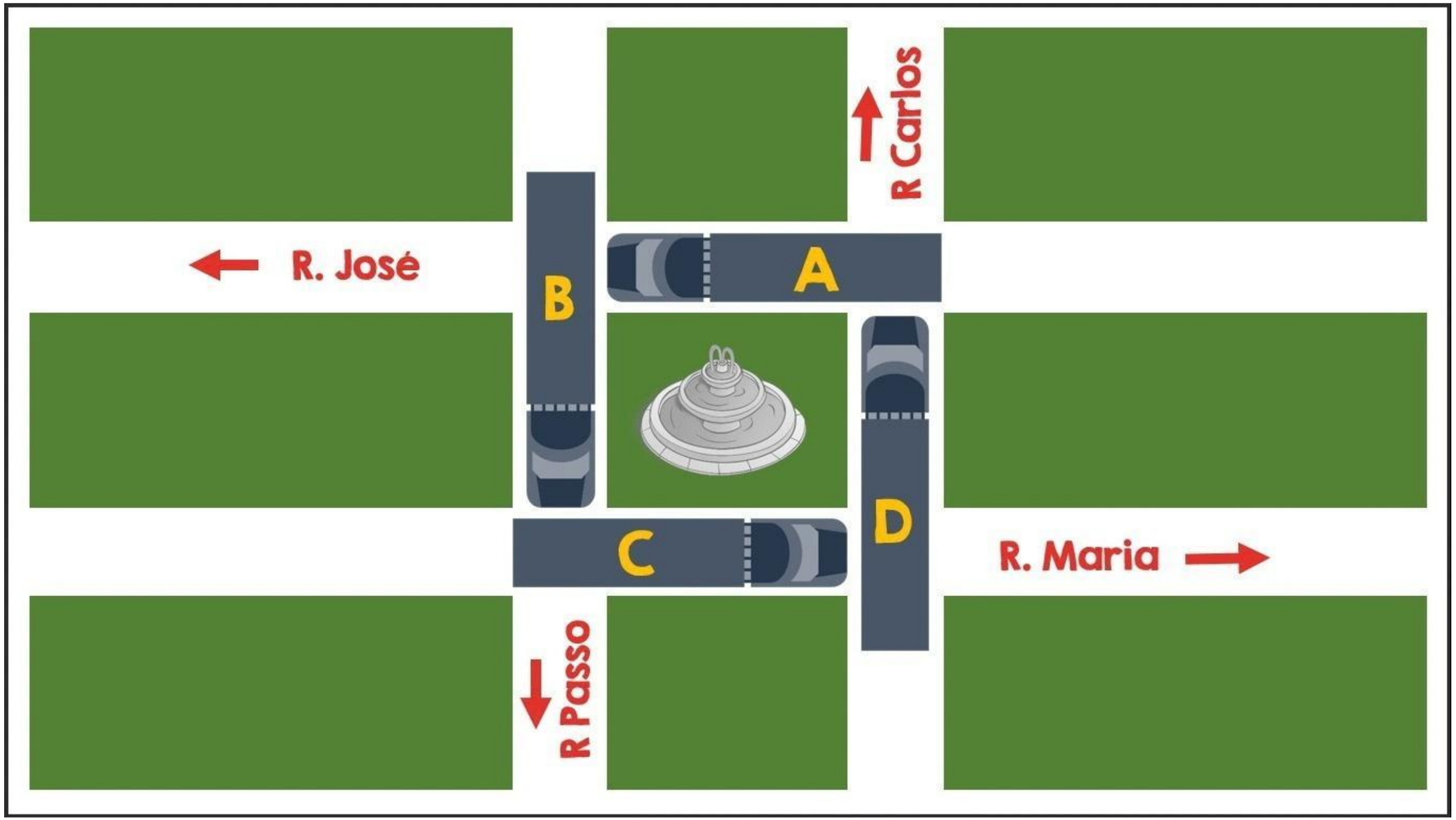
Deadlocks e Inanição

A implementação de um semáforo com fila de espera pode resultar em uma situação em que dois ou mais processos fiquem esperando indefinidamente por um evento que pode ser causado somente por um dos processos em espera.

O evento em questão é a execução de uma operação ***signal ( )***. Quando tal estado é alcançado, dizemos que esses processos estão em ***deadlock***.



Deadlocks e Inanição





Deadlocks e Inanição

Para ilustrar, consideremos um sistema composto por dois processos, ***P0*** e ***P1***, cada um acessando dois semáforos, ***S*** e **Q**, aos quais é atribuído o valor ***1***:

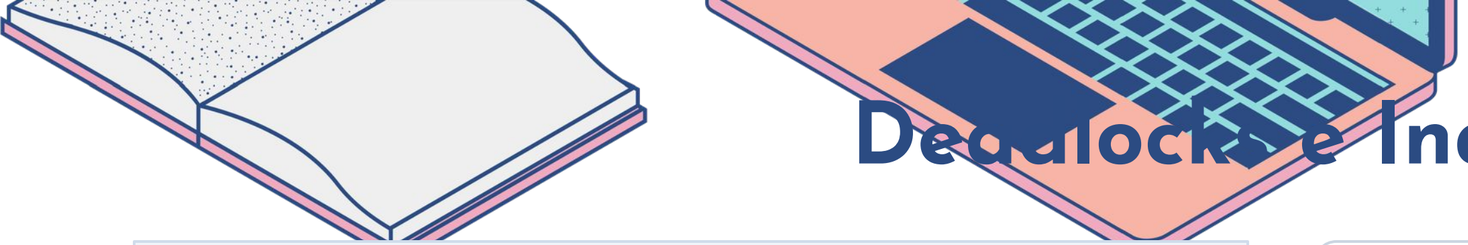
P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);



Deadlocks e Inanição

Suponha que ***P0*** execute ***wait (S)*** e, então, ***P1*** execute ***wait (Q)***. Quando ***P0*** executa ***wait(Q)***, deve esperar até que ***P1*** execute ***signal (Q)***. Da mesma forma, quando ***P1*** executa ***wait(S)***, deve esperar até que ***P0*** execute ***signal(S)***.

P_0	P_1
wait (S);	wait (Q);
wait (Q);	wait (S);
.	.
.	.
.	.
signal (S);	signal (Q);
signal (Q);	signal (S);



Deadlocks e Inanição

O que é Deadlock?

Um conjunto de processos está em estado de **deadlock** quando cada processo está esperando por um evento que só pode ser causado por outro processo do próprio conjunto.

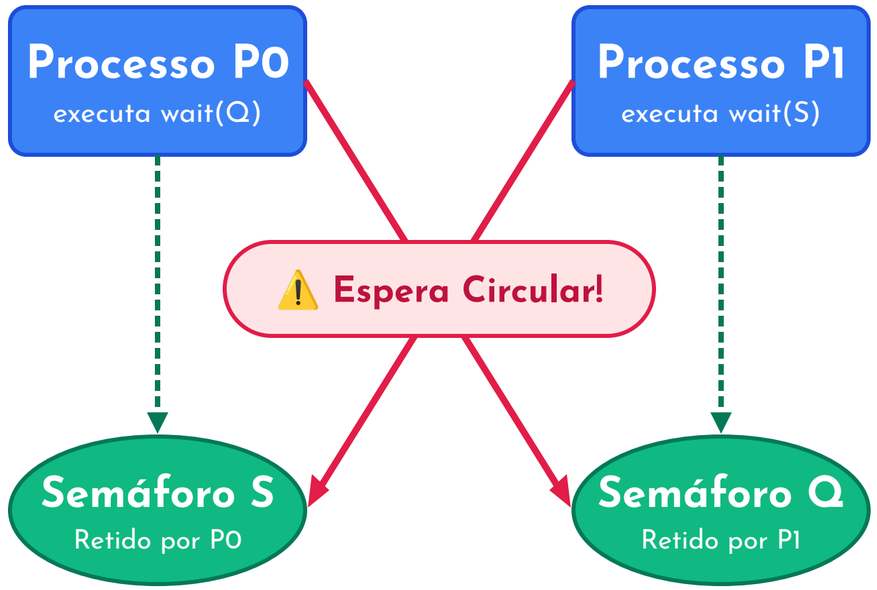
Exemplo: P0 aguarda por ***signal(Q)*** de P1, enquanto P1 aguarda por ***signal(S)*** de P0. Nenhum prossegue!

O que é Inanição (Starvation)?

Também chamada de **bloqueio indefinido**, é a situação em que processos esperam indefinidamente dentro da fila de espera do semáforo.

Causa Comum: Gerenciamento de fila LIFO (último a chegar, primeiro a sair) ou prioridades inadequadas que deixam processos antigos permanentemente sem atendimento.

Diagrama do Ciclo de Deadlock



🔒 **Efeito de Inanição (Starvation)**

Processos ficam bloqueados no semáforo sem conseguir progredir na fila de execução indefinidamente.

Parte V - Monitores

Monitores: Por que Precisamos Deles?

⚠️ O Problema com Semáforos

Embora os semáforos sejam eficazes na sincronização, seu uso incorreto introduz **erros de timing** extremamente difíceis de detectar.

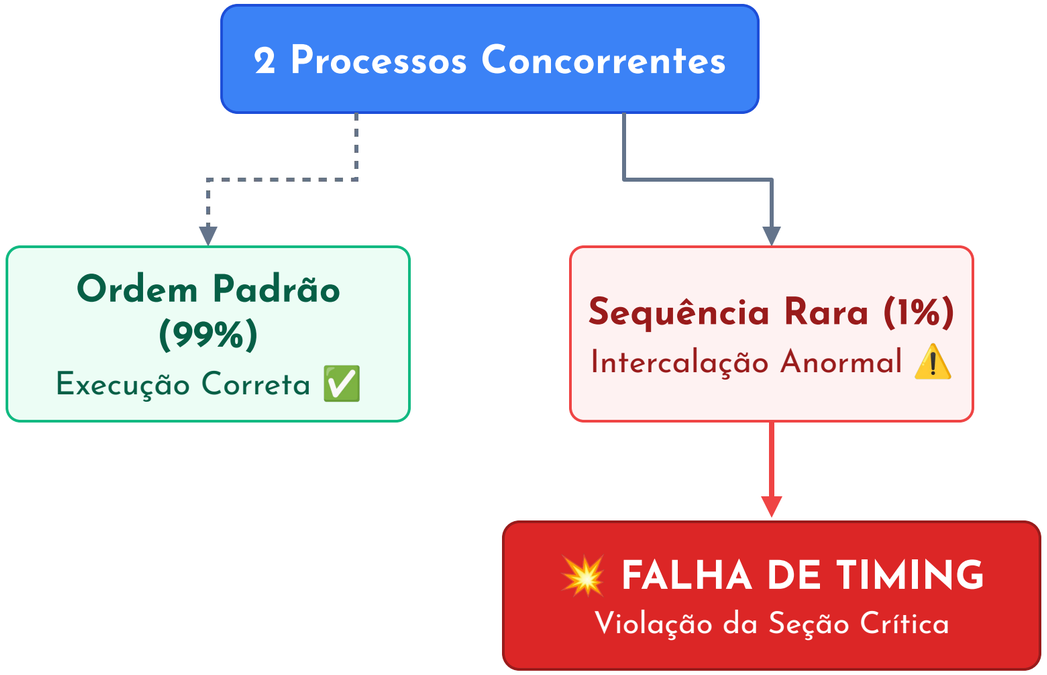
🔍 Exemplo Prático

Se um programador esquecer uma instrução **wait()** ou invertê-la com **signal()**, a seção crítica é violada.

🐛 Por que é tão difícil debugar?

O erro **não acontece sempre**. Ele depende de uma sequência exata de alternância da CPU (não-determinismo).

Diagrama: Falha por Erro de Timing



💡 Por que Monitores resolvem isso?

Monitores encapsulam a sincronização em alto nível, eliminando o risco do programador errar chamadas de semáforos manuais.

Sincronização com Semáforos (Mutex)

📌 Regras de Funcionamento

1. Variável Compartilhada

Todos os processos compartilham a variável **mutex = 1**.

2. Protocolo de Acesso

Cada processo deve obrigatoriamente executar:

🔒 **wait(mutex)** → Antes da Seção Crítica

🔓 **signal(mutex)** → Depois da Seção Crítica

⚠️ Risco de Violação

Se a sequência for desrespeitada, a exclusão mútua falha e múltiplos processos entram na seção crítica ao mesmo tempo!

💡 Exemplo de Fluxo Correto vs Erro

✅ Sequência Correta de Execução

```
wait(mutex);  // mutex passa de 1 para 0
[ Seção Crítica ]  // Apenas 1 processo por vez
signal(mutex);  // Libera acesso (mutex = 1)
```

💥 Cenário de Falha (Erro de Sequência)

Se um processo executar `signal()` antes do `wait()` ou esquecer o `wait()`:

🚨 DOIS PROCESSOS NA SEÇÃO CRÍTICA SIMULTANEAMENTE!

*Essa fragilidade do programador é a principal motivação para a criação dos **Monitores**.*

Monitores: Origem dos Problemas de Sincronização

⚠️ Vulnerabilidade do Sistema

As dificuldades de sincronização surgirão no sistema **mesmo que apenas um único processo** não se comporte adequadamente.

🔍 Causas Principais:

1. Erro de Programação Inocente:
Um bug não intencional, esquecimento ou troca de instruções no código.

2. Programador Não Cooperativo:
Violação consciente/propositada do protocolo de acesso compartilhado.

💡 **Conclusão:** A segurança do sistema não pode depender da "boa vontade" de cada programador.

Diagrama: Efeito de um Único Processo Errado



🛡️ **Monitores** resolvem isso ao centralizar o controle de acesso e impedir execuções arbitrárias!

Monitores: Erro de Inversão das Operações

📌 O Problema da Inversão

Ocorre quando um processo executa erroneamente a operação **signal()** antes de entrar e **wait()** ao sair.

⚠️ Consequência Direta

A **Exclusão Mútua é violada**! O sinal libera a trava precocemente, permitindo acesso simultâneo.

🔍 Impacto na Prática

Vários processos entram na Seção Crítica ao mesmo tempo, causando corrupção de dados e bugs difíceis de reproduzir.

💡 Fluxo de Execução Incorreto

`signal(mutex);`

❌ Libera o acesso antes de entrar!

⬇️

`[ SEÇÃO CRÍTICA ]`

🚨 **MÚLTIPLOS PROCESSOS AQUI DENTRO!**

⬇️

`wait(mutex);`

❌ Tenta bloquear somente na saída!

💥 **RESULTADO: Falha crítica na sincronização!**

Monitores: Substituição Incorreta de signal() por wait()

📌 O Erro de Troca de Função

Ocorre quando o programador substitui accidentalmente a liberação **signal(mutex)** por outro bloqueio **wait(mutex)** ao sair da seção crítica.

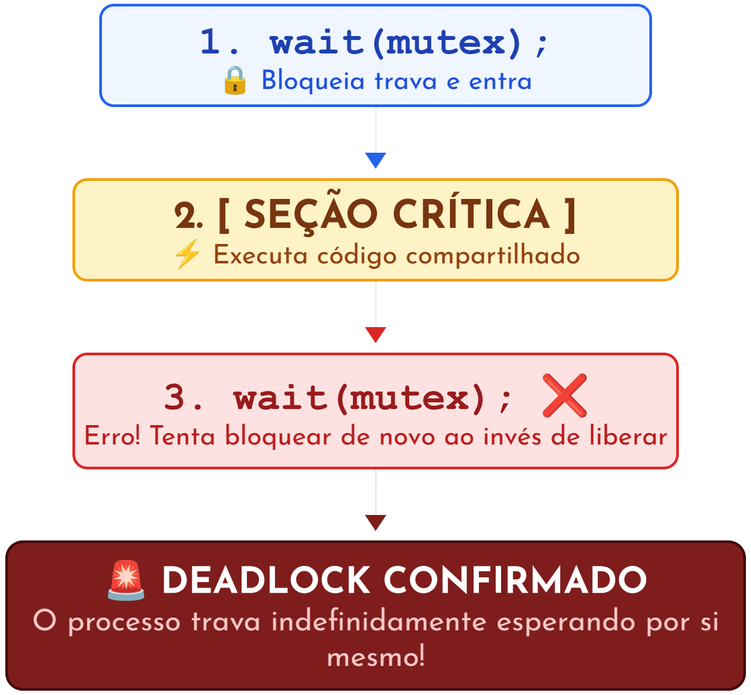
⚠️ Consequência Direta

Um Deadlock Ocorrerá! O processo tenta adquirir novamente uma trava que ele mesmo já possui e não liberou, congelando a execução.

🔍 Análise do Comportamento

Ao invés de sinalizar a outros processos que a região está livre, o processo entra em espera eterna no segundo `wait()`.

Diagrama: Sequência da Falha (Deadlock)



🛡️ **Monitores** evitam esse tipo de bug encapsulando o sinal e a espera nas operações do objeto!

Uso do Monitor: Conceitos Fundamentais

📌 O que é um Monitor?

É um **Tipo Abstrato de Dados (TAD)** que encapsula variáveis compartilhadas, procedimentos de acesso e código de inicialização.

🔒 Exclusão Mútua Garantida

Garante que apenas **um processo por vez** execute dentro do monitor. A sincronização é tratada pelo compilador, reduzindo erros humanos.

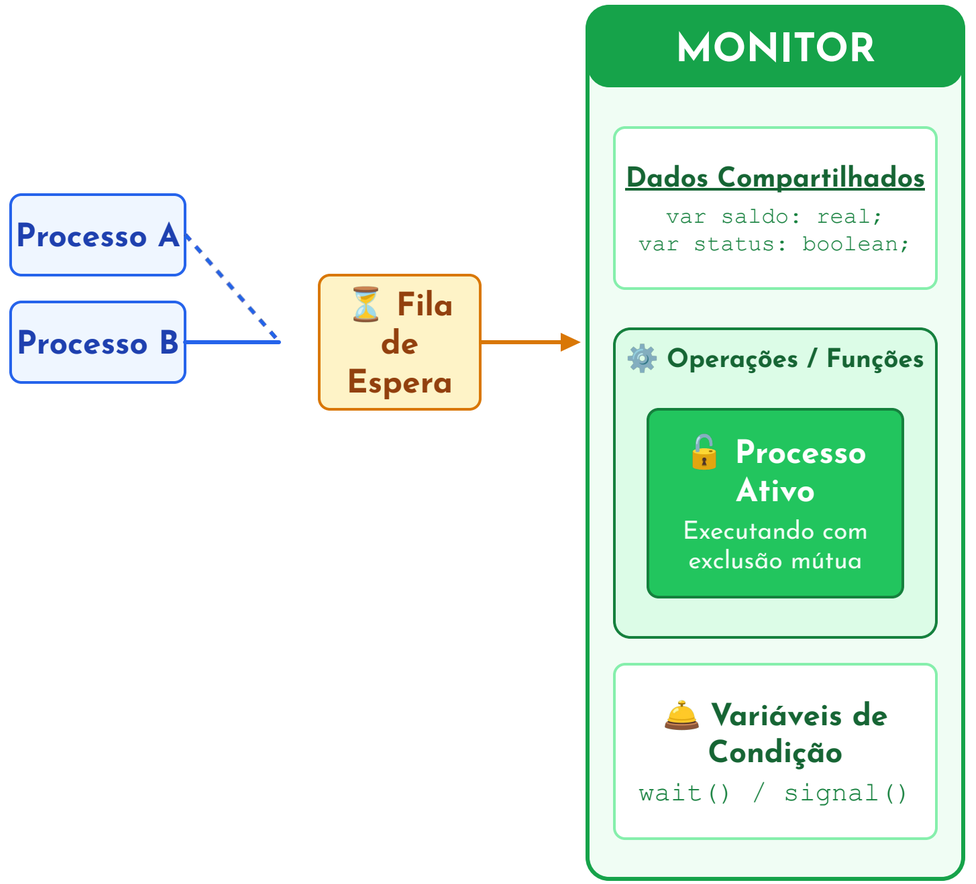
🛠️ Linguagens de Alto Nível

Implementado de forma estruturada em linguagens modernas (ex: o bloco `synchronized` em Java).

💡 Exemplo de Analogia:

Funciona como um **provador de loja com chave interna**. Apenas uma pessoa entra por vez; as outras aguardam na fila até a chave ser liberada.

🔍 Diagrama Arquitetural do Monitor



Uso do Monitor: Regiões Críticas e Compilador

📋 1. Estrutura Interna do Monitor

Um monitor é definido especificando-se um **nome**, **variáveis locais**, **procedimentos** e um **código de inicialização**.

💡 **Exemplo:** Em Java, uma classe com métodos `synchronized` encapsula o estado e garante acesso seguro.

🤖 2. Exclusão Mútua Automática

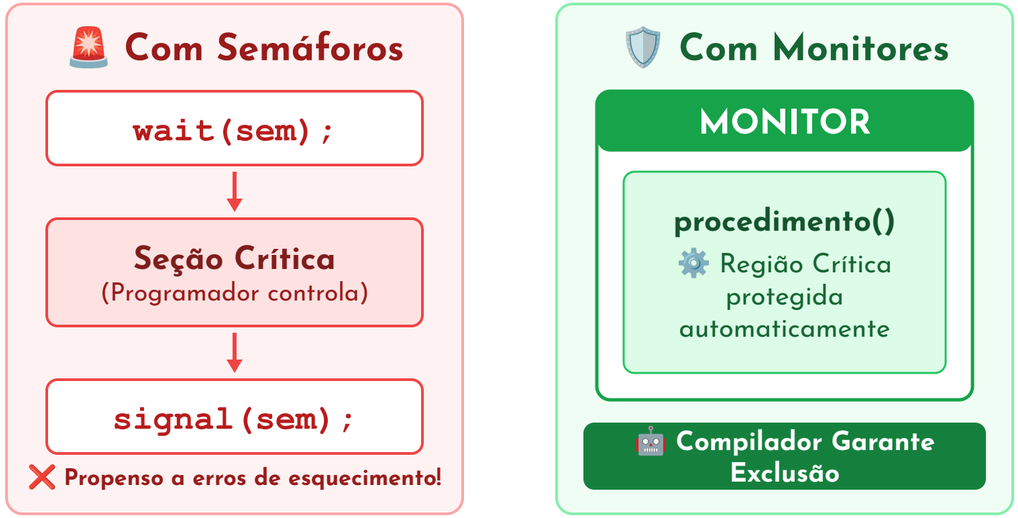
A implementação da exclusão mútua **não é realizada pelo programador** (diferente dos semáforos), evitando travamentos e erros manuais.

⚠️ **Semáforos vs Monitores:** Semáforos exigem `wait/signal` manuais; Monitores automatizam essa trava via compilador.

🔒 3. Procedimentos Protegidos

As regiões críticas são definidas como **procedimentos no monitor**. O compilador garante que apenas um processo execute por vez.

📊 Diagrama: Compilador vs. Programador



🔑 Resumo Teórico para Provas e Concursos:

- **Onde fica a trava?** Encapsulada dentro do próprio objeto/monitor.
- **Quem gerencia os bloqueios?** O compilador insere o código de sincronização automaticamente.
- **Qual a maior vantagem?** Elimina erros de esquecimento de liberação de chave (deadlocks).

Uso do Monitor: Estrutura e Sintaxe

💻 Pseudocódigo Estruturado

```
monitor ContaBancaria {
  /* 1. Variáveis Privadas */
  private double saldo;

  /* 2. Procedimentos Públicos */
  public deposito(valor) {
    saldo = saldo + valor;
  }

  public saque(valor) {
    saldo = saldo - valor;
  }

  /* 3. Inicialização */
  inicializacao() {
    saldo = 0.0;
  }
}
```

📘 Componentes do Monitor

📦 1. Variáveis Compartilhadas

Guardam o estado interno. São **privadas** e não podem ser acessadas diretamente por processos externos.

⚙️ 2. Funções / Procedimentos

Definem as operações permitidas. O compilador garante que **apenas 1 processo por vez** execute qualquer procedimento.

🚀 3. Código de Inicialização

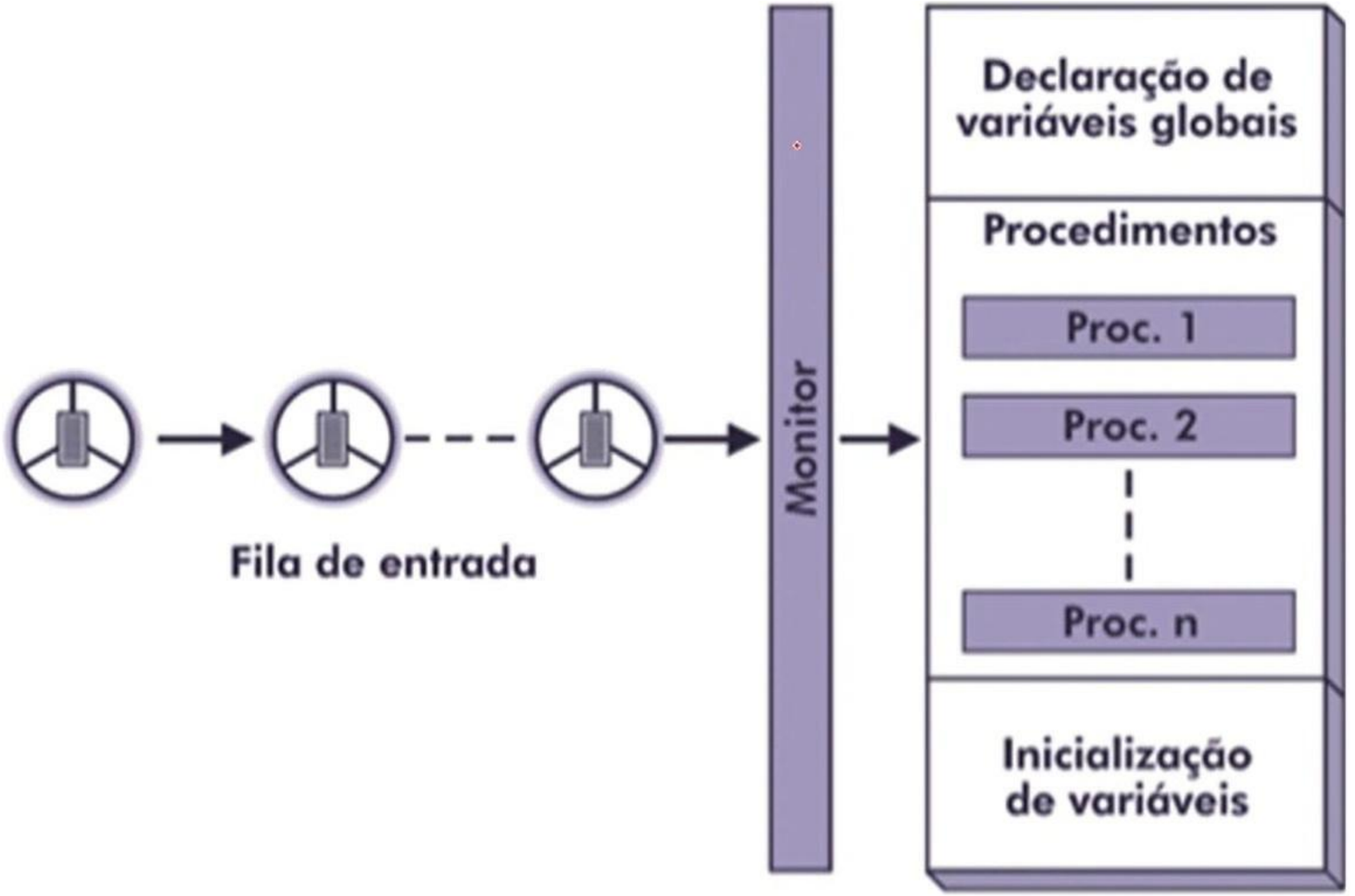
Executado uma única vez na criação do monitor para configurar os valores iniciais das variáveis.

💡 Exemplo Prático (Conta Bancária):

Se dois clientes tentarem fazer um **saque** ao mesmo tempo, o monitor bloqueia um deles até que o outro termine, evitando saldo negativo incorreto.



Uso do Monitor



Transição: De Monitores ao Jantar dos Filósofos

Abstração & Sincronização

O que vimos: Monitores

• **Exclusão Mútua Automática:** Gerenciada pelo compilador, reduzindo erros humanos.

• **Encapsulamento:** Dados privados protegidos por procedimentos públicos.

• **Variáveis de Condição:** Sincronização avançada através de wait() e signal().

Outras Soluções de Condição de Corrida:

✔ **Semáforos:** Sinais inteiros (primitiva de baixo nível).

✔ **Solução de peterson:** Blocos de exclusão mútua explícitos - Espera Ocupada.

APLICAÇÃO PRÁTICA



Desafio Clássico

O Próximo Passo: Filósofos

• **O Problema do Jantar dos Filósofos:** Um cenário clássico modelado por Edsger Dijkstra para testar algoritmos de sincronização.

• **Múltiplos Recursos Compartilhados:** Não basta garantir a exclusão mútua simples; é preciso gerenciar vários recursos simultâneos (garfos).

• **Novos Riscos de Concorrência:**

🚨 **Deadlock (Impasse):** Todos esperam indefinidamente.

🚨 **Starvation (Inanição):** Um processo nunca obtém os recursos.

"O jantar dos Filósofos"



O Jantar dos Filósofos (Problema Clássico de Sincronização)

Regras do Cenário

- **5 Filósofos** sentados em uma mesa redonda.
- Existe apenas **1 garfo** entre cada par de filósofos vizinhos (total de 5 garfos).
- Para comer espaguete, o filósofo precisa de **2 garfos** (o da esquerda e o da direita).
- Cada filósofo alterna continuamente entre duas tarefas: **Pensar** ou **Comer**.

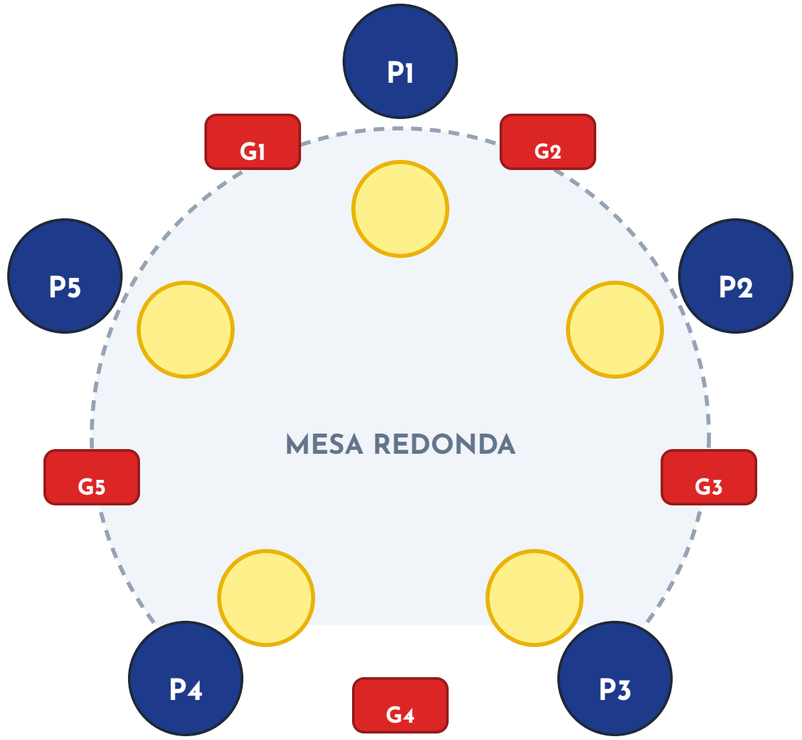
Exemplo Prático de Disputa

Passo 1: Desejo de Comer

Se o **Filósofo P1** quer comer, ele tenta pegar o **Garfo G1** (esquerda) e o **Garfo G2** (direita). Em qualquer ordem.

Passo 2: O Conflito (Bloqueio)

Se o vizinho **P2** já pegou o **Garfo G2**, o filósofo **P1** fica **travado aguardando** o garfo ser liberado!



Legenda Diagrama: P = Filósofos | G = Garfos (recursos compartilhados) | Pratos Amarelos = Espaguete

O Jantar dos Filósofos | O Desafio do Algoritmo

O Problema em Questão

Por definição, o objetivo central é responder à seguinte pergunta:

"Você é capaz de propor um algoritmo que implemente cada filósofo de modo que ele execute as tarefas de comer e pensar sem nunca ficar travado?"

Desafio Tecnológico:

Cada filósofo representa um **processo/thread** concorrente disputando **recursos compartilhados** (os garfos) em um sistema operacional.

Condições para Uma Boa Solução

1. Sem Deadlock (Impasse Total)

Nenhum cenário pode fazer com que todos os filósofos fiquem esperando eternamente com 1 garfo na mão.

2. Sem Starvation (Inanição)

Nenhum filósofo pode morrer de fome esperando indefinidamente pela sua vez de comer.

3. Alta Concorrência

Filósofos vizinhos não podem comer ao mesmo tempo, mas filósofos não vizinhos devem poder comer simultaneamente.

O Jantar dos Filósofos | (1) Solução Naïve & Risco de Falha

Como Funciona essa Solução?

A abordagem mais simples é tentar pegar um garfo de cada vez:

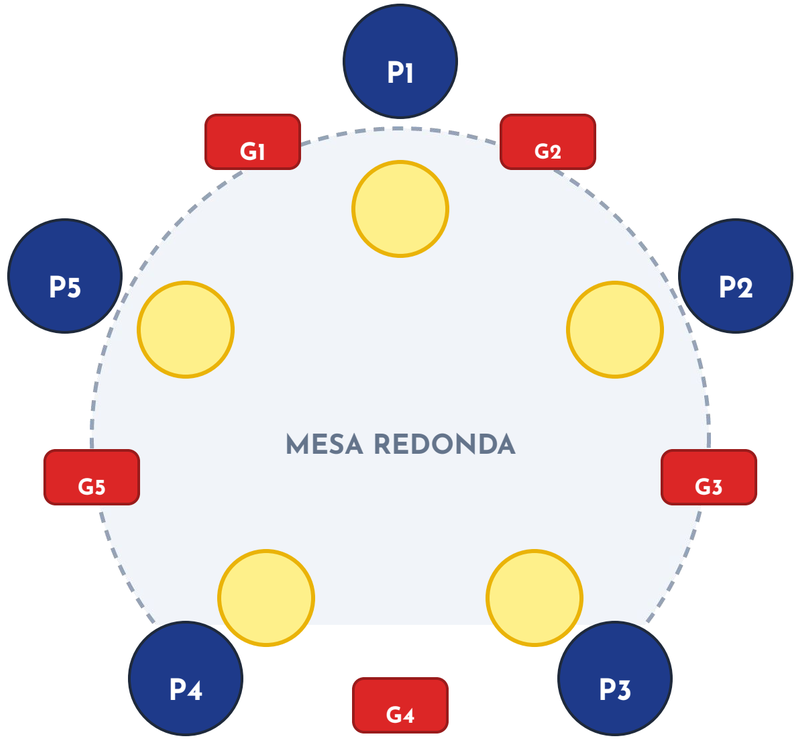
1. Pegar Garfo 1 (Esquerda)
Tenta pegar o garfo da esquerda. Se não conseguir, fica aguardando.

2. Pegar Garfo 2 (Direita)
Em seguida, tenta pegar o garfo da direita.

3. Comer e Devolver
Com os dois garfos na mão, come o espaguete e devolve um de cada vez.

Pseudocódigo do Filósofo:

```
while(true) {
pensar();
pegar_garfo(esquerda);
pegar_garfo(direita);
comer();
liberar_garfos();
}
```



Legenda Diagrama: P = Filósofos | G = Garfos (recursos compartilhados) | Pratos Amarelos = Espaguete

O Jantar dos Filósofos | (1) Solução Naïve & Risco de Falha

Como Funciona essa Solução?

A abordagem mais simples é tentar pegar um garfo de cada vez:

1. Pegar Garfo 1 (Esquerda)
Tenta pegar o garfo da esquerda. Se não conseguir, fica aguardando.

2. Pegar Garfo 2 (Direita)
Em seguida, tenta pegar o garfo da direita.

3. Comer e Devolver
Com os dois garfos na mão, come o espaguete e devolve um de cada vez.

Pseudocódigo do Filósofo:

```
while(true) {
pensar();
pegar_garfo(esquerda);
pegar_garfo(direita);
comer();
liberar_garfos();
}
```

O Risco Fatal: Deadlock (Impasse)

Cenário Crítico de Falha:

Se os **5 filósofos** sentarem ao mesmo tempo e cada um pegar o garfo da sua **esquerda simultaneamente**:

Estado do Sistema após a Falha:

- **Todos possuem 1 garfo** na mão esquerda.
- **Todos aguardam** o garfo da direita ser liberado pelo vizinho.
- **Ninguém solta** o garfo que já possui.

RESULTADO: BLOQUEIO PERMANENTE

O sistema entra em **Deadlock**. Os filósofos morrem de fome aguardando infinitamente.

O Jantar dos Filósofos | (2) Solução com Liberação & Risco de Livelock

A Lógica da Solução

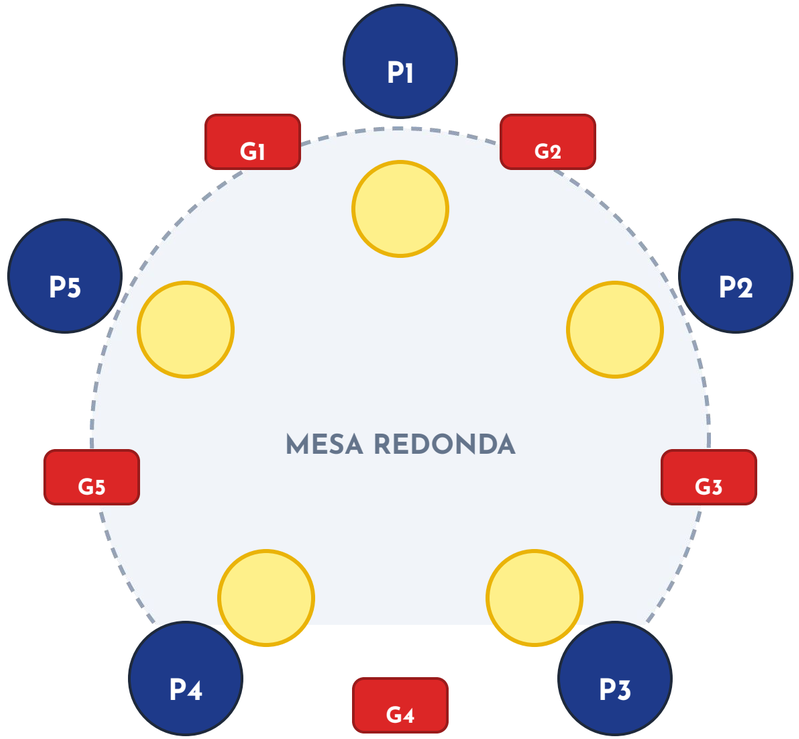
Para evitar o travamento (Deadlock), se o filósofo não conseguir o segundo garfo, ele **solta o primeiro** e aguarda um tempo fixo:

1. Tenta pegar Garfo 1 (Esquerda) - Conseguiu!
2. Tenta pegar Garfo 2 (Direita)
3. Se Garfo 2 estiver ocupado:

- Devolve o Garfo 1 à mesa imediatamente
- Aguarda um **tempo fixo (ex: 5s)** e tenta tudo de novo

Algoritmo Modificado:

```
while(true) {
pegar_garfo(esquerda);
if (!pegar_garfo(direita)) {
soltar_garfo(esquerda);
esperar(5_segundos);
continue;
}
comer(); liberar_garfos();
}
```



Legenda Diagrama: P = Filósofos | G = Garfos (recursos compartilhados) | Pratos Amarelos = Espaguete

O Jantar dos Filósofos | (2) Solução com Liberação & Risco de Livelock

A Lógica da Solução

Para evitar o travamento (Deadlock), se o filósofo não conseguir o segundo garfo, ele **solta o primeiro** e aguarda um tempo fixo:

1. Tenta pegar Garfo 1 (Esquerda) - Conseguiu!
2. Tenta pegar Garfo 2 (Direita)
3. Se Garfo 2 estiver ocupado:

- Devolve o Garfo 1 à mesa imediatamente
- Aguarda um **tempo fixo (ex: 5s)** e tenta tudo de novo

Algoritmo Modificado:

```
while(true) {
pegar_garfo(esquerda);
if (!pegar_garfo(direita)) {
soltar_garfo(esquerda);
esperar(5_segundos);
continue;
}
comer(); liberar_garfos();
}
```

O Novo Problema: Livelock (Impasse Ativo)

Exemplo de Cenário Sincronizado:

Se **todos os 5 filósofos** executarem a ação exatamente ao mesmo tempo, o sistema entra em loop infinito sem travar!

Passo 1: Todos pegam o garfo da esquerda simultaneamente.
Passo 2: Ninguém consegue o da direita e todos soltam o garfo da esquerda juntos.
Passo 3: Todos esperam os mesmos **5 segundos** e repetem tudo de novo!

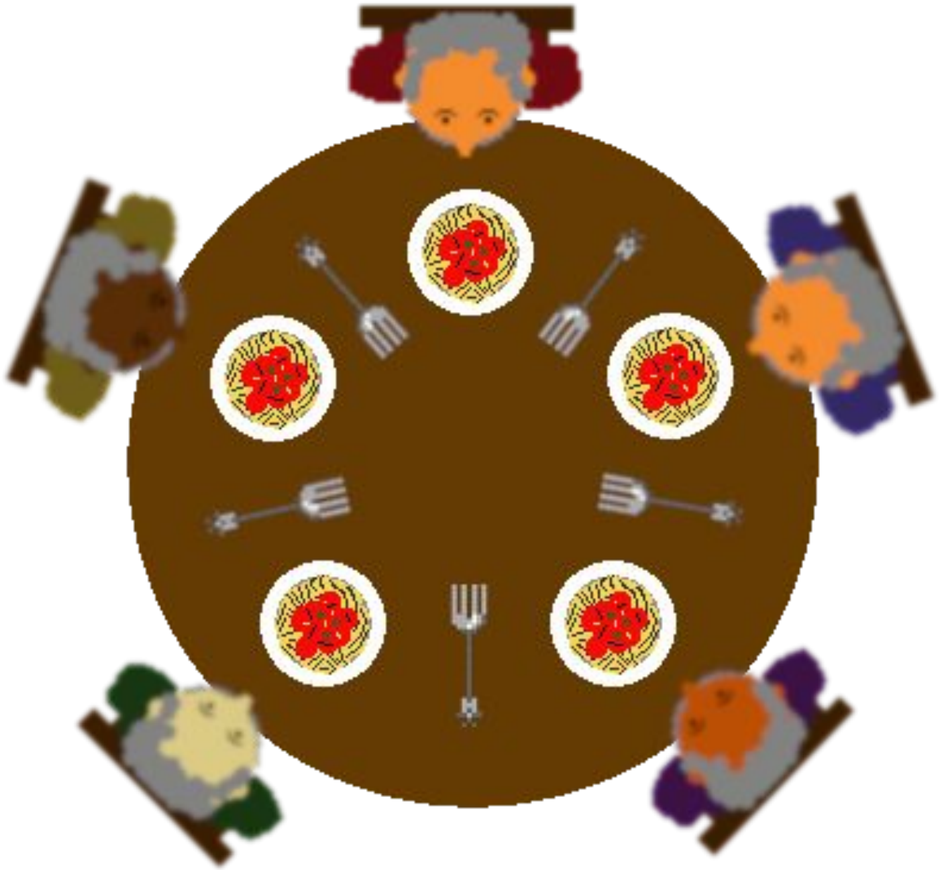
RESULTADO: LIVELOCK

Os processos continuam ativos mudando de estado, mas **nenhum progride ou consegue comer.**



"O jantar dos Filósofos"

Vocês podem propor uma solução melhor?



O Jantar dos Filósofos | (3) Solução por Hierarquia de Garfos

Como Funciona a Hierarquia?

Atribui-se uma enumeração linear aos garfos (G1 a G5).
Todo filósofo deve pegar primeiro o garfo de **menor índice**.

1. Regra de Ouro (Ordem de Índices):

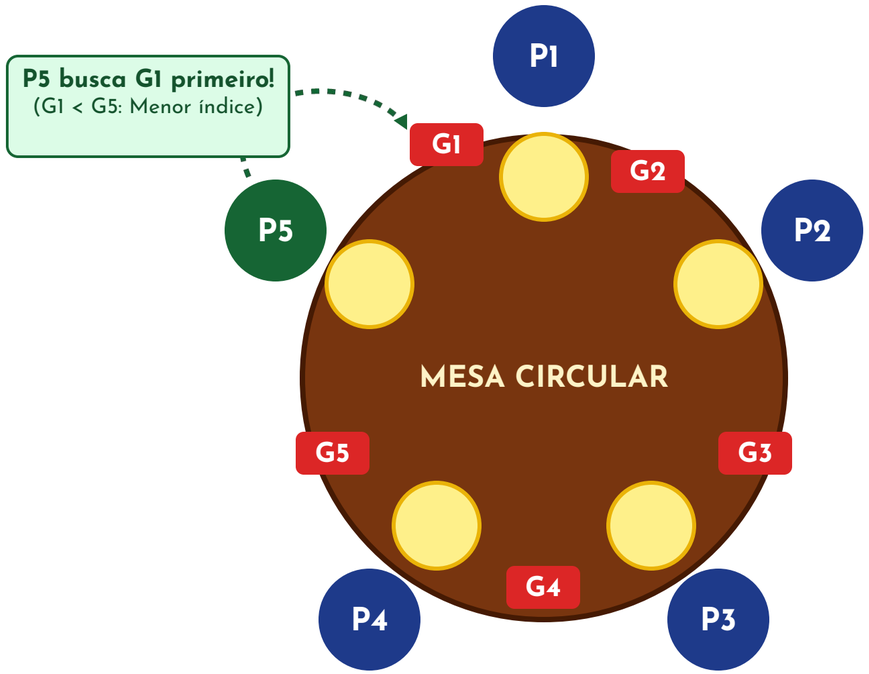
Sempre solicita primeiro **min(Garfo_Esq, Garfo_Dir)** e depois **max(Garfo_Esq, Garfo_Dir)**.

2. Quebra de Simetria (Exemplo com P5):

P1 a P4 pegam o garfo da esquerda primeiro. Porém, **P5 (entre G5 e G1) tenta pegar G1 primeiro**! Se G1 estiver ocupado, P5 espera sem segurar o G5.

Pseudocódigo do Filósofo:

```
primeiro = min(esquerda, direita);
segundo = max(esquerda, direita);
pegar_garfo(primeiro);
pegar_garfo(segundo);
comer(); liberar_garfos();
```



RESULTADO: LIVRE DE DEADLOCK!

A dependência circular é quebrada. Pelo menos um filósofo sempre consegue ambos os garfos e o sistema progride com segurança!